

Department of Electronic and Telecommunication Engineering
University of Moratuwa, Sri Lanka



System Serial Bus Design

EN4021 - Advanced Digital Systems

MEMBERS:	INDEX No
Thuvaragan S.	210657G
Sundarbavan T.	210624E
Wijethilaka U.G.P.M.B.	210724K

December 10, 2025

Contents

1 Overview	3
1.1 Architecture	3
1.2 Key Features	3
2 Address Allocation	3
2.1 General Address Encoding	3
2.2 Without Bus Bridge	3
2.3 With Bus Bridge	5
3 Master Port	5
3.1 Purpose and role	5
3.2 Block Diagram	5
3.3 I/O Description	6
3.4 State Diagram and Behaviour	6
4 Slave Port	7
4.1 Purpose	7
4.2 Block Diagram	7
4.3 I/O Description	7
4.4 State Diagram	8
5 Arbiter	8
5.1 Overview	8
5.2 I/O Description	8
5.3 State Machine	9
6 Address Decoder	9
6.1 Purpose	9
6.2 I/O Description	9
6.3 Block Diagram	10
6.4 State Diagram	10
7 Bus Interconnect	11
7.1 Module	11
7.2 I/O Description	12
8 Bus Bridge	12
8.1 Bus Bridge Master	12
8.2 I/O Description	13
8.3 State Machine Logic	13
8.4 Bus Bridge Slave	13
8.5 I/O Description	14
8.5.1 Bridge Slave State Machine	14
9 UART Module	14
9.1 I/O Description	15
10 Timing Analysis Report	15
11 Resource Utilization Report	16
12 Simulation	16
12.1 Verification Strategy	16
12.2 Testbench: master2_slave3_tb.sv	16

12.3	Arbiter verification simulations	17
12.3.1	Simple Arbiter verification	17
12.3.2	Arbiter split scenario verification	17
12.3.3	Arbiter full verification	18
12.4	Address decoder simulation	18
12.5	Top Level verification simulations	19
12.5.1	Single Master Request (Write)	19
12.5.2	Single Master Request (Read)	19
12.5.3	Two masters Request	20
12.5.4	Split Transaction	20
Appendix		22

1 Overview

The Serial Bus System is a compact, resource-efficient serial bus designed for FPGA platforms. The implementation reported here targets the Intel Cyclone IV family on the Terasic DE0-Nano board and demonstrates a multi-master multi-slave serial interconnect supporting split transactions and a UART-based bus bridge for inter-bus communication.

The system uses a 1-bit serial data path and an explicit request/grant arbitration scheme with split-transaction support to allow long-latency peripherals to relinquish the bus while the rest of the system continues operation.

1.1 Architecture

The implemented design supports:

- Two masters (Master 1: high priority, Master 2: low priority/Bridge Master).
- Three slaves (Slave 1 and Slave 2 local BRAM-backed slaves; Slave 3 acting as a Bus Bridge).
- Central Bus Interconnect containing: Arbiter, Address Decoder, Master/Slave multiplexers and control logic.
- Split transaction support for slaves that require long response latency (specifically Slave 3 in Bridge mode).
- UART-based bus bridge to enable inter-FPGA communication.

1.2 Key Features

- **Multi-Master Support:** Two masters with deterministic priority arbitration (M1 prioritized). Master 2 serves as the interface for remote commands via UART.
- **Split Transactions:** Slaves that cannot respond immediately (like the Bridge Slave) can assert a `ssplit` line. The arbiter tracks the split owner and issues a `split_grant` when the slave becomes ready, allowing other masters to use the bus in the interim.
- **Bus Bridge:** UART-based bridge supports translation between local serial bus frames and remote bus frames; uses FIFOs to cross clock domains.
- **Resource Efficiency:** Single-bit serial data and compact FSMs reduce LUT/logic usage and routing complexity.

2 Address Allocation

2.1 General Address Encoding

A 16-bit address format is used in the implemented design. The upper bits (4 MSB (bits [15:12])) contain the *Device ID* that selects one of the slaves. The remaining low-order bits index the memory inside the slave.

2.2 Without Bus Bridge

When the bridge is not enabled (local-only operation), the address space is partitioned as follows:

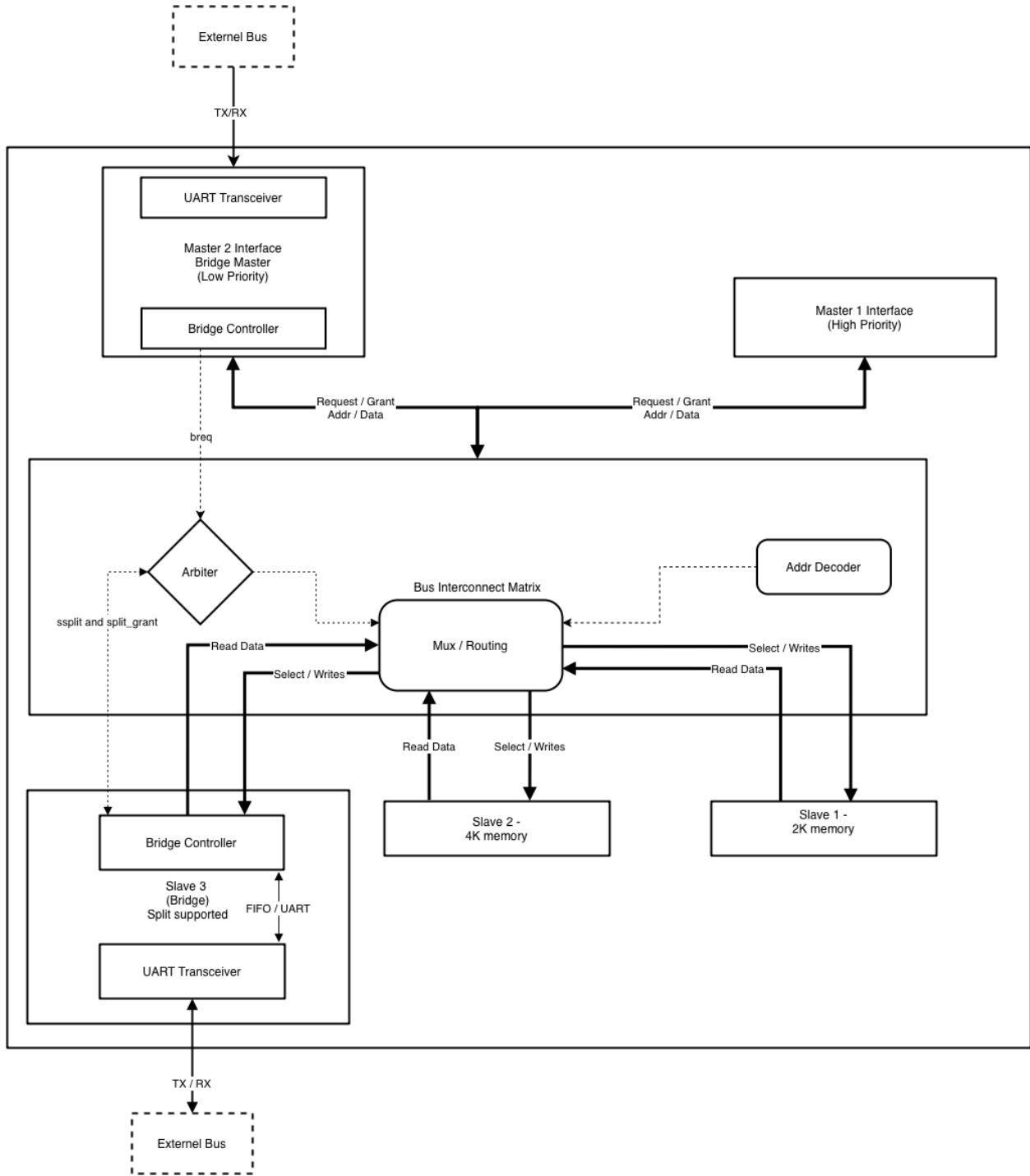


Figure 1: Full System Architecture

Table 1: Local Address Map

Device ID	Slave	Size	Address Range
0000	Slave 1	2 KB	0x0000 – 0x07FF
0001	Slave 2	4 KB	0x1000 – 0x1FFF
0010	Slave 3	4 KB	0x2000 – 0x2FFF

2.3 With Bus Bridge

When Slave 3 is configured as the bus bridge, addresses mapped into Slave 3's window (0x2000–0x2FFF) are translated and forwarded to a remote bus via UART. The bridge aggregates the remote slave memory and exposes it through this window.

3 Master Port

3.1 Purpose and role

The Master Port module provides the interface between a local master device and the serial bus. It serializes addresses and data LSB-first ordering, asserts bus requests and reacts to grants, and handles split/resume handshake.

3.2 Block Diagram

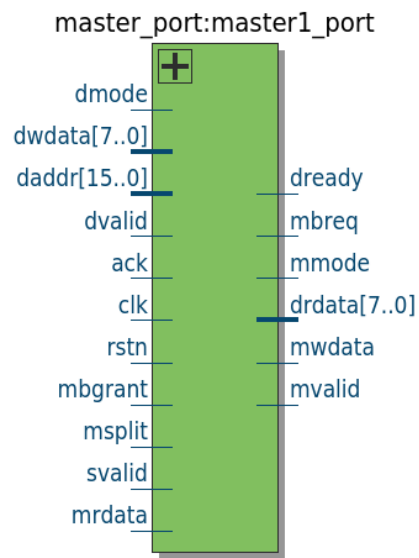


Figure 2: Master Port

3.3 I/O Description

Table 2: Master Port I/O

Signal	Dir	Description
daddr[15:0]	Input	16-bit address from master device
dwdata[7:0]	Input	8-bit write data from master device
drdata[7:0]	Output	8-bit read data returned to master device
dvalid	Input	Master asserts to request a transaction
dready	Output	Port ready for new transaction (IDLE)
dmode	Input	0=Read, 1=Write
mwdata	Output	Serial bit stream to bus (address/data)
mrdata	Input	Serial bit received from a slave
mmode	Output	Serial mode to bus (0=Read, 1=Write)
mvalid	Output	Serial valid signal pointing to bus
svalid	Input	Valid signal from slave (during read)
mbreq	Output	Bus request line to arbiter
mbgrant	Input	Bus grant from arbiter
msplit	Input	Indicates current transaction is split
ack	Input	Acknowledge from address decoder

3.4 State Diagram and Behaviour

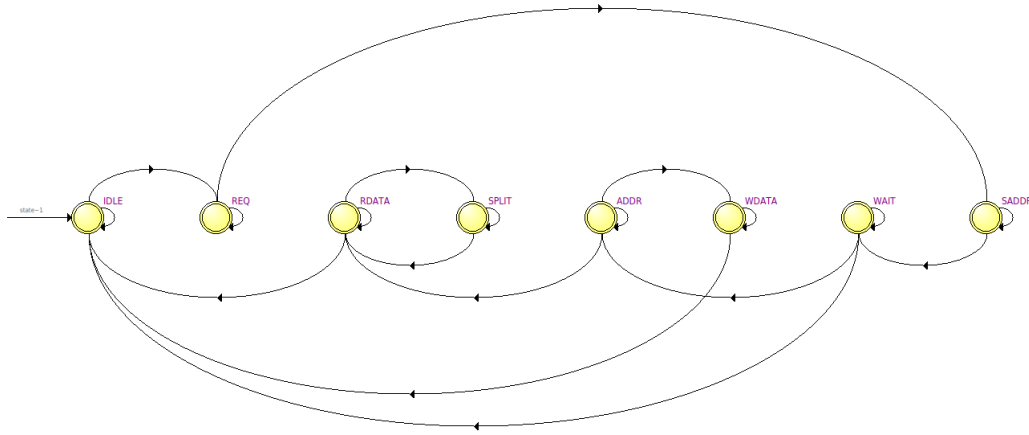


Figure 3: Master Port State Machine

The Master FSM acts as the initiator of all transactions. As shown in Figure 3, the operation proceeds as follows:

- **IDLE**: The master waits for a valid request signal (**dvalid**) from the core logic.
- **REQ**: Upon request, it asserts **mbreq** and waits in this state until the Arbiter grants access (**mbgrant**).
- **SADDR**: The master serializes the 4-bit Device Address (MSB-first) to select the target slave.
- **WAIT**: It waits for the Address Decoder to assert **ack**, confirming the slave is present and ready.
- **ADDR**: If acknowledged, it serializes the 12-bit Memory Address (LSB-first).

- **WDATA / RDATA:** Depending on the mode, it either transmits write data (WDATA) or waits to receive read data (RDATA).
- **SPLIT:** If the slave asserts `ssplit`, the master enters the SPLIT state and releases the bus (de-asserts `mbreq`), allowing other masters to use it. It remains here until the Arbiter re-grants the bus via `mbgrant`.

4 Slave Port

4.1 Purpose

The Slave Port module deserializes incoming address/data, controls a local BRAM (or a peripheral), and serializes read data back to the requesting master. It handles the `ssplit` protocol for high-latency operations.

4.2 Block Diagram

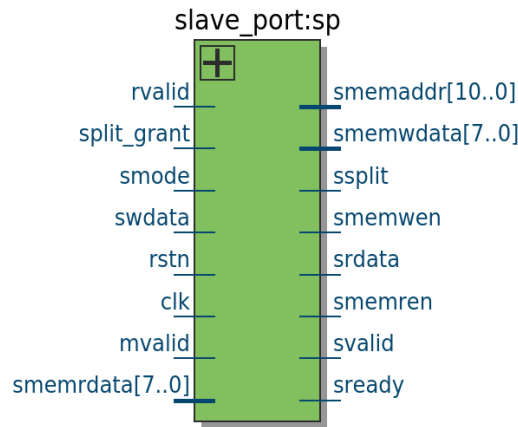


Figure 4: Slave Port

4.3 I/O Description

Table 3: Slave Port I/O

Signal	Dir	Description
<code>swdata</code>	Input	Serial stream from the active master
<code>srdata</code>	Output	Serial read data bitstream to the master
<code>mvalid</code>	Input	Valid indicates valid bits coming from master
<code>sready</code>	Output	Indicates slave ready to accept new transaction
<code>svalid</code>	Output	Read-data valid signal
<code>ssplit</code>	Output	Slave asserts to request split
<code>split_grant</code>	Input	Arbiter grants resumed access for split owner
<code>smemaddr</code>	Output	Address to the local memory/peripheral
<code>smemrdata</code>	Input	Read data from memory/peripheral
<code>smemwen</code>	Output	Memory write enable
<code>smemren</code>	Output	Memory read enable

4.4 State Diagram

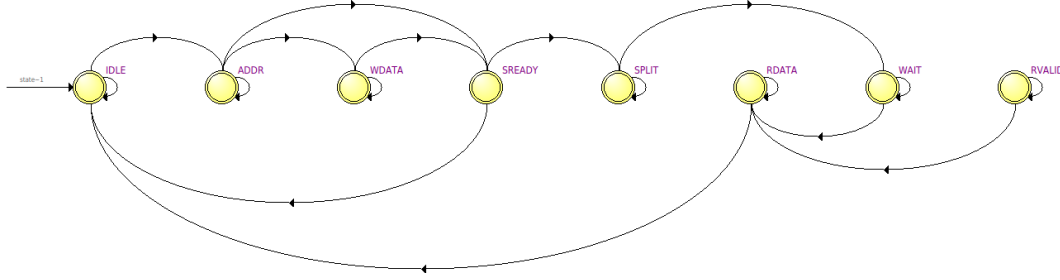


Figure 5: Slave Port State Machine

The Slave Port FSM (Figure 5) manages deserialization and memory access:

- **IDLE**: Waits for the `mvalid` signal indicating the start of a transaction.
- **ADDR**: Deserializes the incoming address bits into a register.
- **SREADY**: Once the address is received, it asserts `sready` to the local memory controller to latch the address/data.
- **WDATA**: If writing, it deserializes the data bits.
- **RDATA**: If reading, it serializes the data from memory back to the bus.
- **SPLIT**: For the Bridge Slave, it may enter this state to signal the Arbiter that the response is pending (UART latency).

5 Arbiter

5.1 Overview

The arbiter serializes access to the shared serial bus, giving Master 1 priority over Master 2. It also manages split transactions, tracking the split-owner and re-granting the bus when the slave is ready.

5.2 I/O Description

Table 4: Arbiter I/O

Signal	Dir	Description
<code>breq1</code>	Input	Bus request from Master 1 (High Priority)
<code>breq2</code>	Input	Bus request from Master 2 (Low Priority)
<code>bgrant1</code>	Output	Grant signal to Master 1
<code>bgrant2</code>	Output	Grant signal to Master 2
<code>split_grant</code>	Output	Signal to all masters that split phase is ending
<code>sready1/2</code>	Input	Ready signals from Slaves 1 and 2
<code>sreadysp</code>	Input	Ready signal from Split Slave (S3)
<code>ssplit</code>	Input	Split request signal from Slave 3
<code>msplit1/2</code>	Output	Indicates to masters that transaction is split

5.3 State Machine

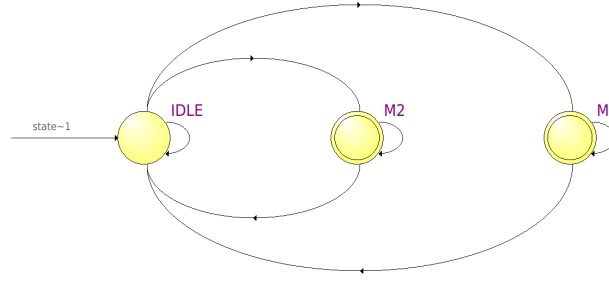


Figure 6: Arbiter State Machine

The Arbiter uses a priority-based FSM (Figure 6) to manage bus ownership:

- **IDLE:** Checks for requests. M1 has strict priority over M2.
- **M1 / M2:** Grants the bus to the respective master.
- **Split Handling:** If a transaction splits, the Arbiter remembers the "Split Owner". It allows the non-owner to use the bus (IDLE state) but prevents them from accessing the specific slave involved in the split. When the slave signals completion (**split_grant**), the Arbiter re-grants the bus to the original owner.

6 Address Decoder

6.1 Purpose

The address decoder samples the serial device ID bits and asserts **ssel** and **ack** to select the correct slave. It validates the address against the map.

6.2 I/O Description

Table 5: Address Decoder I/O

Signal	Dir	Description
mwdata	Input	Serial data stream from Master (contains address)
mvalid	Input	Valid signal from Master
ack	Output	Acknowledge signal to Master
ssel[1:0]	Output	Slave Select lines (Binary encoded)
mvalid1/2/3	Output	Routed valid signal to selected slave
sready1/2/3	Input	Ready status from slaves
ssplit	Input	Split request input
split_grant	Input	Split grant input from Arbiter

6.3 Block Diagram

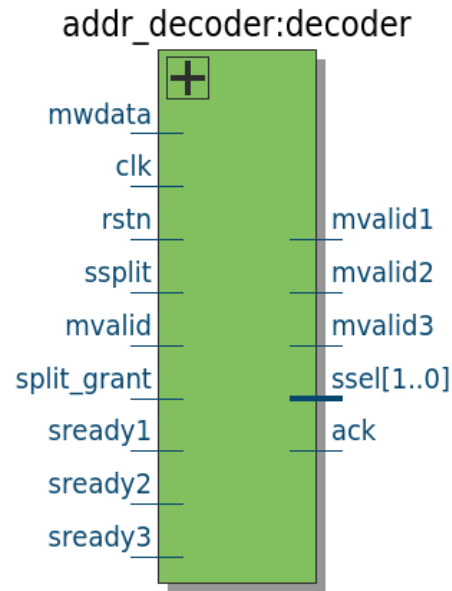


Figure 7: Address Decoder

6.4 State Diagram

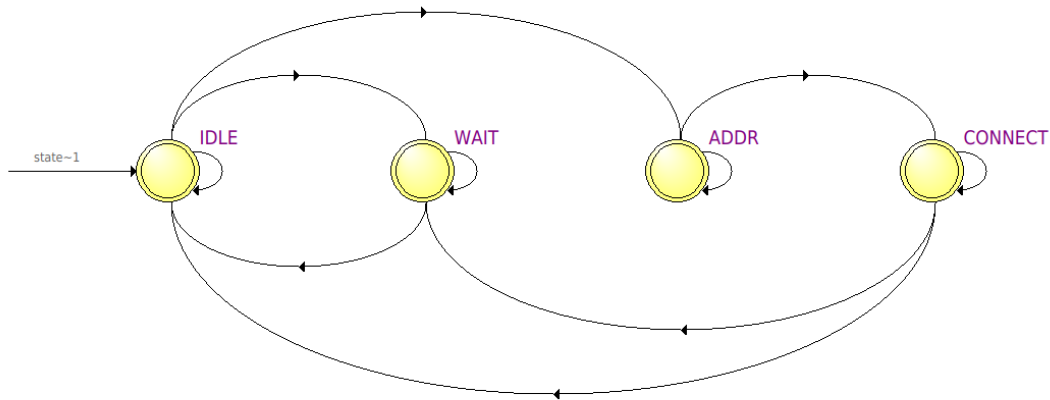


Figure 8: Address Decoder State Machine

The Address Decoder FSM (Figure 8) synchronizes with the Master's addressing phase:

- **IDLE**: Ready for start of frame.
- **ADDR**: Receive 4-bit Device ID.

- **CONNECT:** Validates the ID. If valid (Device 0, 1, or 2) and the slave is ready, it asserts **ack**.
- **WAIT:** Holds the connection open for the duration of the master's validity signal.

7 Bus Interconnect

7.1 Module

The `bus_m2_s3.v` module instantiates the arbiter, decoder, and multiplexers. It defines the point-to-point routing between the active master (selected by Arbiter) and active slave (selected by Decoder).

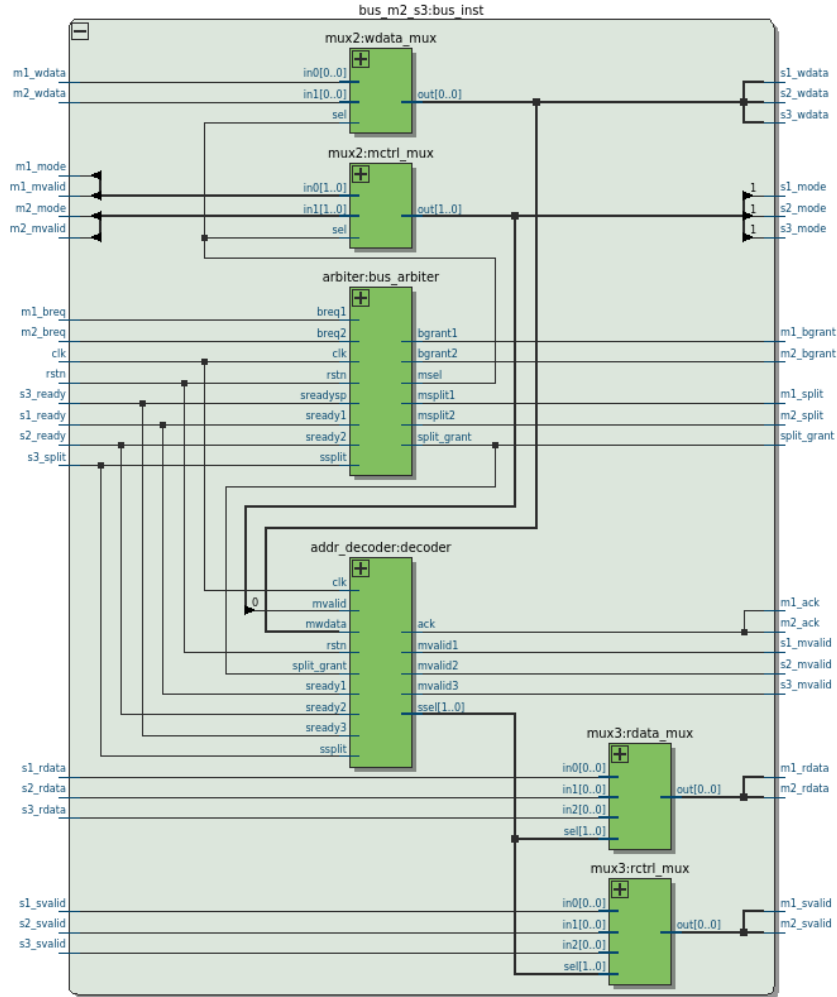


Figure 9: Bus Interconnect

7.2 I/O Description

Table 6: Bus Interconnect (Top Level) I/O

Signal	Dir	Description
m1_rdata	Output	Serial read data to Master 1
m1_wdata	Input	Serial write data/addr from Master 1
m1_mbreq	Input	Request from Master 1
m1_mbgrant	Output	Grant to Master 1
m2_*	In/Out	Similar interface for Master 2
s1_wdata	Output	Serial write data/addr to Slave 1
s1_rdata	Input	Serial read data from Slave 1
s3_ssplrit	Input	Split signal from Slave 3

8 Bus Bridge

8.1 Bus Bridge Master

The Bridge Master listens for UART commands from a remote system and issues requests to the local bus. It uses a FIFO to buffer incoming commands (transiting from UART baud clock to Bus clock).

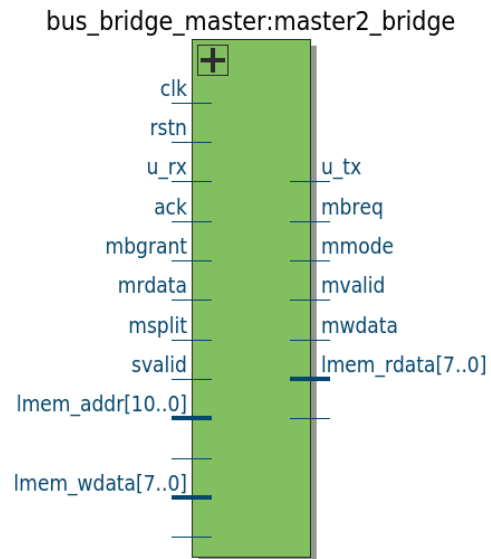


Figure 10: Bus Bridge Master

8.2 I/O Description

Table 7: Bus Bridge Master I/O

Signal	Dir	Description
u_rx	Input	UART Receive line (Commands from remote)
u_tx	Output	UART Transmit line (Responses to remote)
mwdata	Output	Serial data to bus
mrdata	Input	Serial data from bus
mbreq	Output	Bus Request
lmem_*	In/Out	Interface to local BRAM

8.3 State Machine Logic

The Bus Bridge Master relies on a pipelined control flow rather than a single FSM state diagram. Its operation logic is as follows:

- **Command Input:** Incoming UART bytes are deserialized and pushed into a FIFO buffer.
- **Bus Transaction:** A controller monitors the FIFO. When data is available, it initiates a Master Port transaction by asserting `dvalid` and providing the address/data.
- **Response Forwarding:** For read operations, the controller waits for the Master Port to return valid data (`drdata`) and then triggers the UART transmitter to send this byte back to the remote host.

8.4 Bus Bridge Slave

The Bus Bridge Slave forwards local bus transactions to the remote system via UART. For reads, it uses the split transaction mechanism.

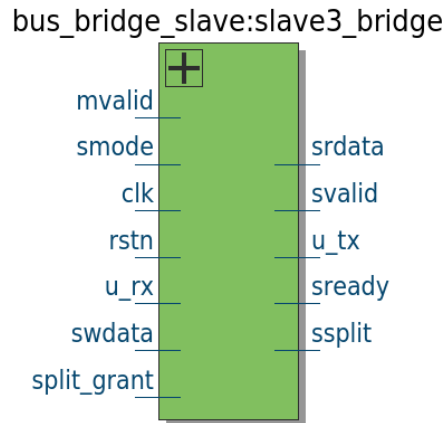


Figure 11: Bus Bridge Slave

8.5 I/O Description

Table 8: Bus Bridge Slave I/O

Signal	Dir	Description
swdata	Input	Serial data from master
srdata	Output	Serial data to master
u_tx	Output	UART Transmit (Forward to remote)
u_rx	Input	UART Receive (Response from remote)
ssplit	Output	Split signal (Asserted during UART latency)
lmem_*	In/Out	Interface to local BRAM

8.5.1 Bridge Slave State Machine

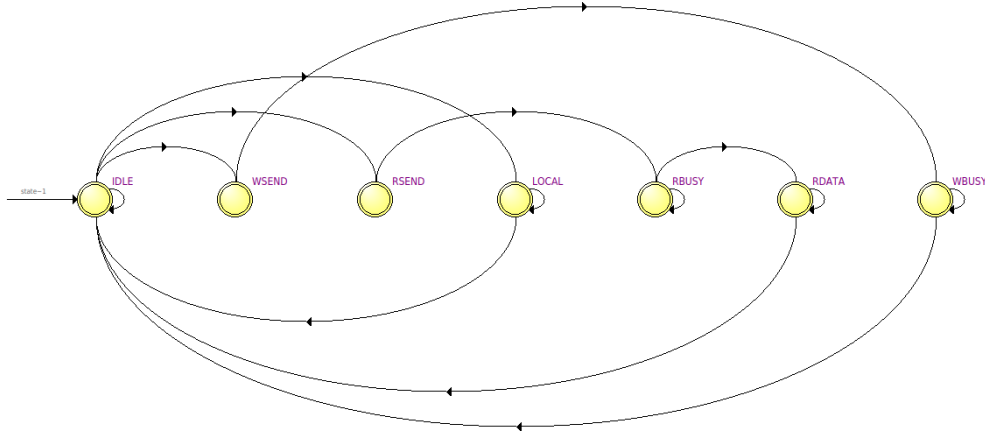


Figure 12: Bus Bridge Slave Controller FSM

The Bridge Slave controller handles the complex coordination between the high-speed bus and the slow UART link (Figure 12):

- **WSEND / RSEND**: Initiates a UART transmission for Write or Read commands.
- **WBUSY / RBUSY**: Waits for the UART transmitter to complete sending the command frame.
- **RDATA**: For read transactions, it waits here for the response byte from the remote system via UART RX. The **ssplit** signal is held high during this entire wait time.
- **LOCAL**: Handles direct access to the local BRAM if the address falls within the local range.

9 UART Module

The system uses a configurable UART for the physical link. It supports 8N1 format and parameterized widths.

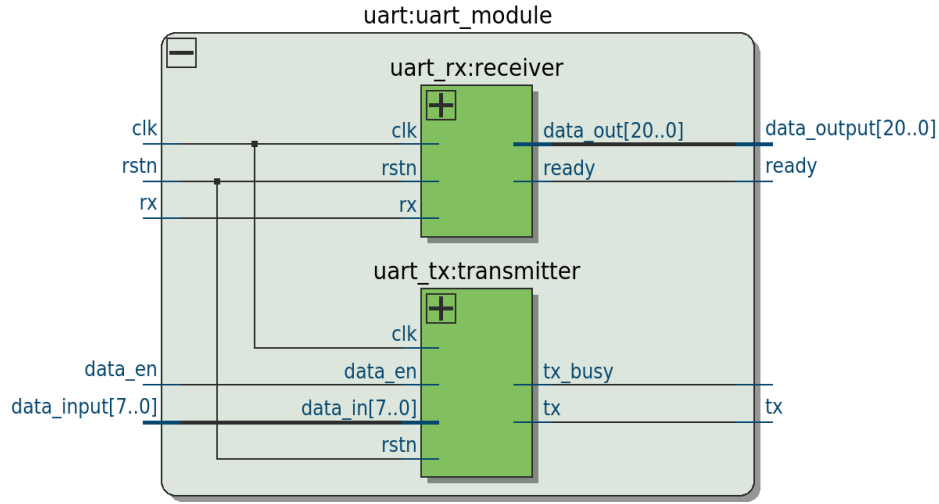


Figure 13: UART Module Schematic

9.1 I/O Description

Table 9: UART Module I/O

Signal	Dir	Description
data_input	Input	Parallel data to transmit
data_en	Input	Enable transmission
tx	Output	Serial Transmit line
rx	Input	Serial Receive line
data_output	Output	Parallel received data
ready	Output	Data received valid signal

10 Timing Analysis Report

The design was synthesized and timing-analyzed for the Cyclone IV (EP4CE22F17C6) with a 50 MHz system clock.

Timing Summary (Slow 85C Model):

- **Setup Slack:** 14.222 ns
- **Hold Slack:** 0.292 ns
- **Recovery Slack:** 16.805 ns
- **Removal Slack:** 2.660 ns

Given a 50 MHz clock (20 ns period) and a setup slack of 14.222 ns, the design has a significant timing margin.

$$T_{min} = 20 \text{ ns} - 14.22 \text{ ns} = 5.78 \text{ ns}$$

$$F_{max} \approx \frac{1}{5.78 \text{ ns}} \approx 173 \text{ MHz}$$

The design easily meets the timing requirements.

11 Resource Utilization Report

Resource usage synthesized for the DE0-Nano target (Cyclone IV EP4CE22F17C6):

Table 10: FPGA Resource Usage (Measured)

Resource Type	Usage / Total
Total Logic Elements	1,551 / 22,320 (7%)
– Combinational Functions	1,255 / 22,320 (6%)
– Dedicated Logic Registers	1,077 / 22,320 (5%)
Total Registers	1,077
Total Pins	19 / 154 (12%)
Total Memory Bits	65,536 / 608,256 (11%)
Total PLLs	0 / 4 (0%)

Notes:

- The memory usage (65,536 bits) corresponds to the instantiated Block RAMs (BRAM) for the slave memories and UART FIFOs.
- Logic utilization is low (<10%), leaving ample room for additional logic or masters.

12 Simulation

12.1 Verification Strategy

The verification process followed a bottom-up approach, starting from unit tests for individual modules and progressing to a comprehensive system-level testbench. The verification pyramid strategy was employed:

1. **Unit Tests:** Individual modules (Arbiter, Decoder, Master/Slave Ports) were verified in isolation.
2. **Integration Tests:** `master2_slave3_tb.sv` verified the interaction between 2 masters and 3 slaves.
3. **System Tests:** Final validation on the DE0-Nano FPGA hardware.

12.2 Testbench: `master2_slave3_tb.sv`

The primary integration testbench, `master2_slave3_tb.sv`, was designed to exhaustively test the bus protocol and arbitration logic. It covers the following scenarios:

- **Single Master Access:** Verifies basic read and write transactions from M1 and M2 individually.
- **Simultaneous Access (Arbitration):** Asserts request signals from both masters simultaneously to verify that Master 1 (Higher Priority) is granted access first, followed by Master 2.
- **Split Transactions:** Targets Slave 3 (Bridge/Split-Capable) to verify that the bus is released (`split` asserted) and re-granted (`split_grant`) correctly.
- **Address Decoding:** random addresses are generated to ensure requests are routed to the correct slave (S1, S2, or S3).
- **Resource Conflicts:** Simulates scenarios where one master writes to an address while another attempts to read from it.

12.3 Arbiter verification simulations

12.3.1 Simple Arbiter verification

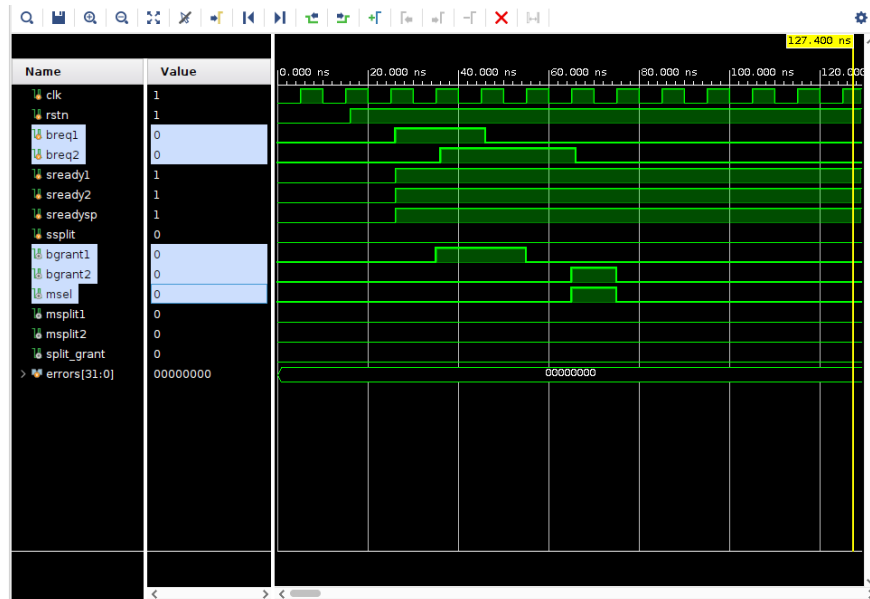


Figure 14: master1 and master2 request bus access

12.3.2 Arbiter split scenario verification

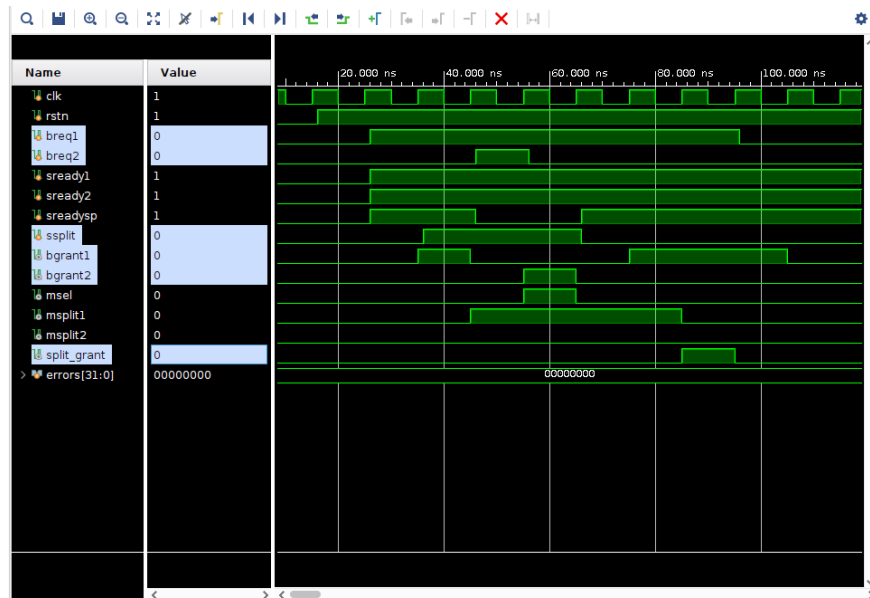


Figure 15: Arbiter handling split transaction

12.3.3 Arbiter full verification

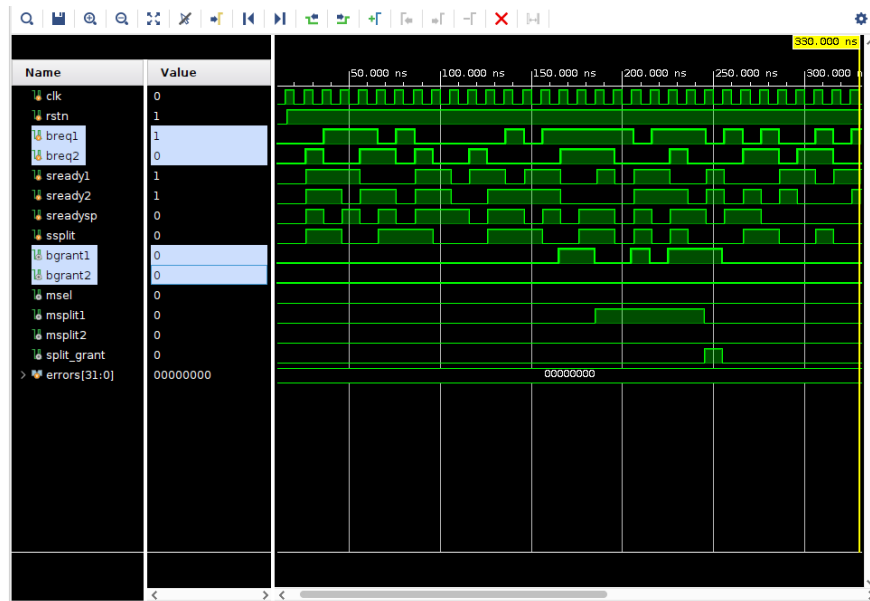


Figure 16: Combinations of bus requests and different slave availability

12.4 Address decoder simulation

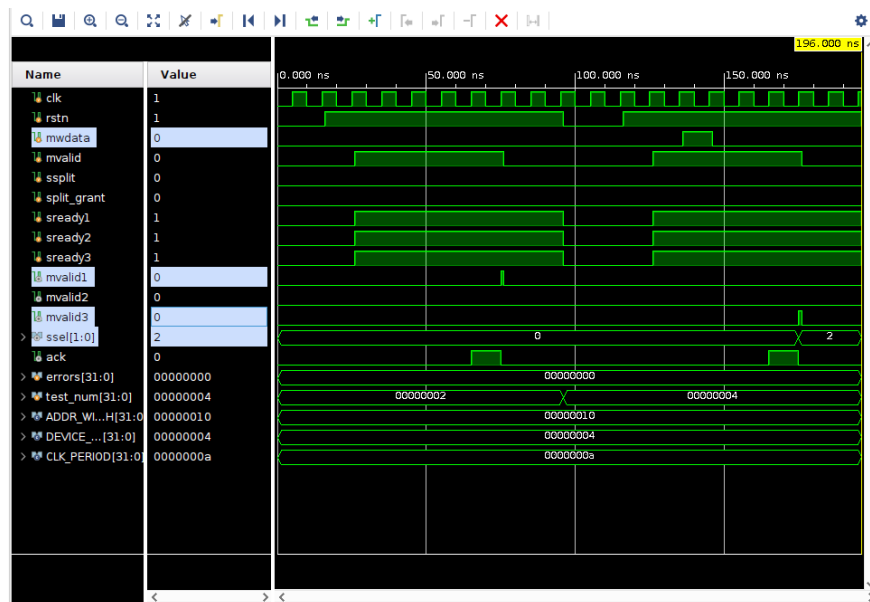


Figure 17: Slave select simulation : selecting slave1 and slave3

12.5 Top Level verification simulations

12.5.1 Single Master Request (Write)

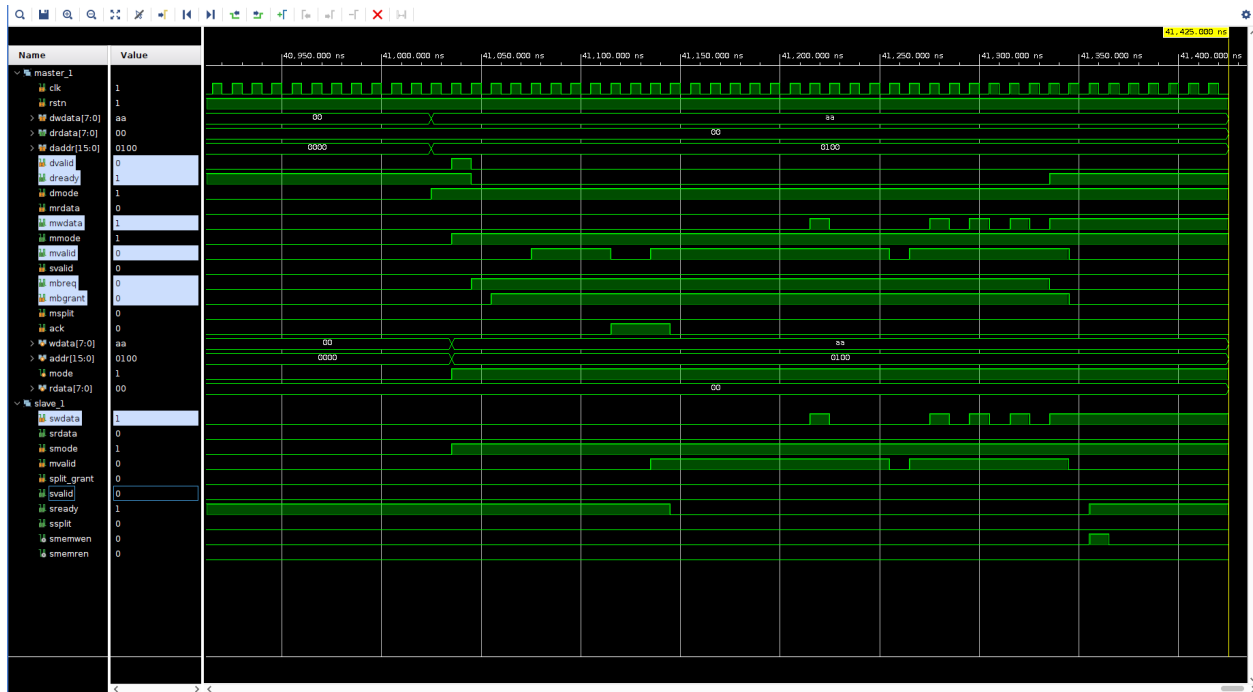


Figure 18: master1 writes to slave1

12.5.2 Single Master Request (Read)

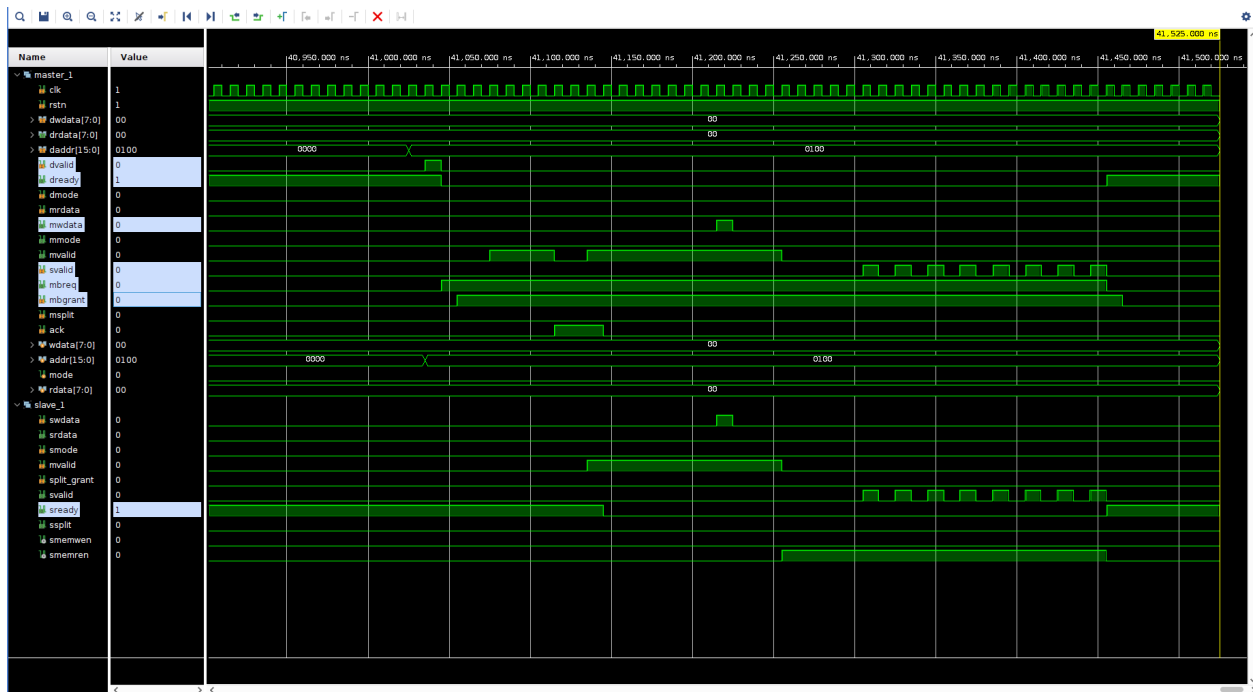


Figure 19: master1 reads from slave1

12.5.3 Two masters Request

Master 1 and Master 2 both send write requests, Master 1 gets priority and Master 2 gets bus access after the Master 1 transaction is complete.

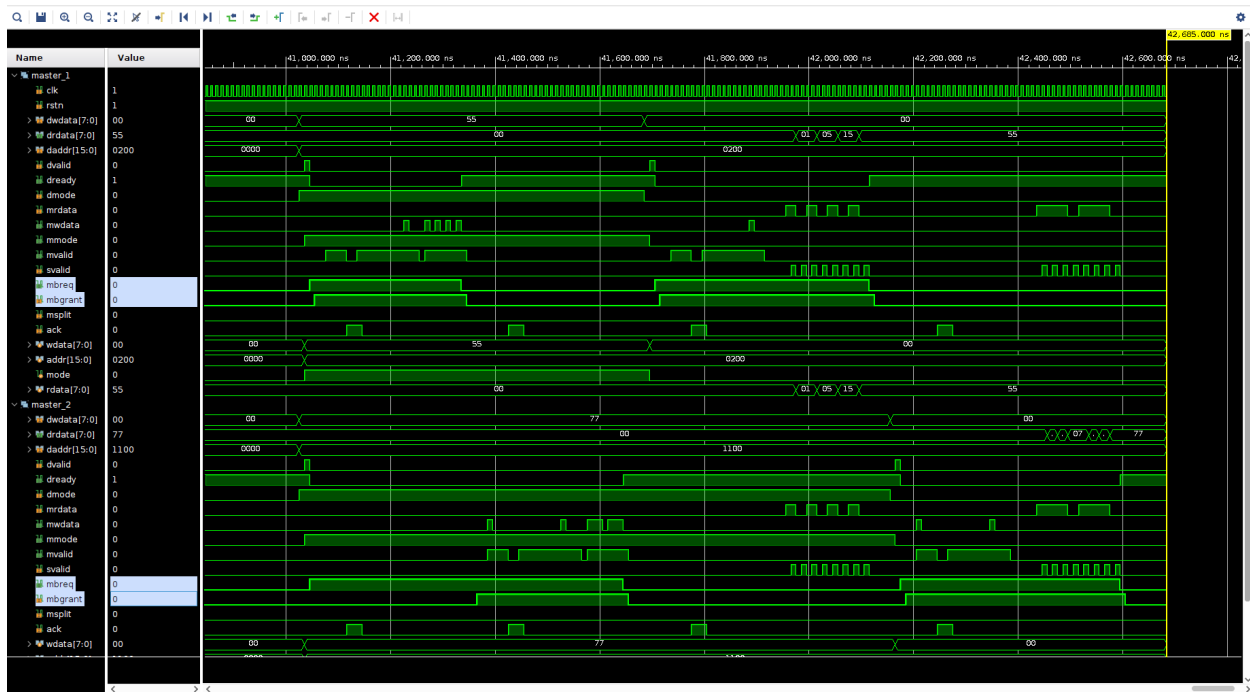


Figure 20: master1 writes to slave1 and master2 writes to slave2 simultaneously

12.5.4 Split Transaction

Master 1 writes to Slave 3 (split-capable). Slave 3 asserts 'split' signal during read. Arbiter releases bus while Slave 3 fetches data. Arbiter grants bus back to Slave 3 when data ready. Master 1 reads back split transaction result.

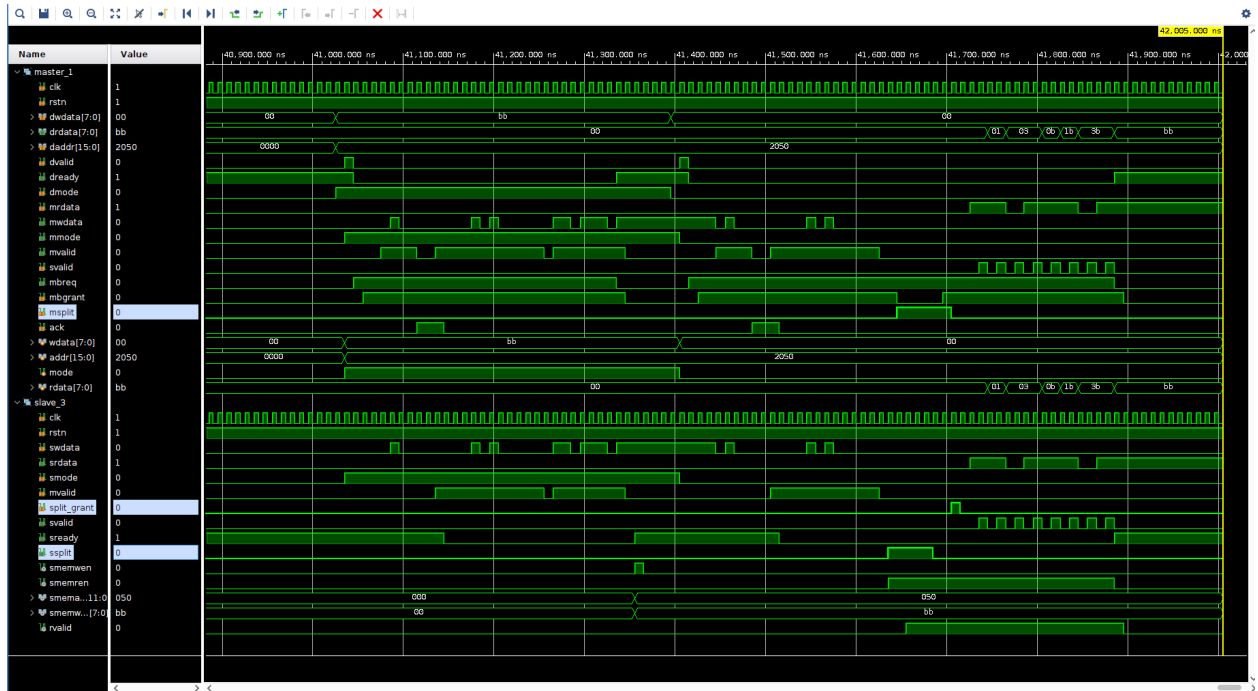


Figure 21: Split transaction between master1 and slave3

Appendix

Verilog Source Code

Master Port (master_port.v)

```
1  `timescale 1ns / 1ps
2
3  module master_port #(
4      parameter ADDR_WIDTH = 16,
5      parameter DATA_WIDTH = 8,
6      parameter SLAVE_MEM_ADDR_WIDTH = 12
7  )(
8      input clk,
9      input rstn,
10
11     // Signals connecting to master device
12     input  [DATA_WIDTH-1:0] dwdata,      // write data
13     output [DATA_WIDTH-1:0] drdata,      // read data
14     input  [ADDR_WIDTH-1:0] daddr,       // address
15     input          dvalid,              // data valid
16     output          dready,             // ready for transaction
17     input          dmode,               // 0 - read, 1 - write
18
19     // Signals connecting to serial bus
20     input          mrdata,              // read data (serial)
21     output reg     mwdata,              // write data and address (serial)
22     output          mmode,              // 0 - read, 1 - write
23     output reg     mvalid,              // wdata valid
24     input          svalid,              // rdata valid
25
26     // Signals to arbiter
27     output          mbreq,              // bus request
28     input           mbgrant,            // bus grant
29     input           msplit,             // split signal
30
31     // Acknowledgement from address decoder
32     input           ack
33 );
34
35     localparam SLAVE_DEVICE_ADDR_WIDTH = ADDR_WIDTH - SLAVE_MEM_ADDR_WIDTH;
36     localparam TIMEOUT_TIME = 5;
37
38     /* Internal signals */
39     // Registers to accept data from master device and slave
40     reg [DATA_WIDTH-1:0] wdata;
41     reg [ADDR_WIDTH-1:0] addr;
42     reg                 mode;
43     reg [DATA_WIDTH-1:0] rdata;
44
45     // Counters
46     reg [7:0] counter, timeout;
47
48     // States
49     localparam IDLE   = 3'b000,
50                ADDR   = 3'b001,      // Send address to slave
51                RDATA  = 3'b010,      // Read data from slave
52                WDATA  = 3'b011,      // Write data to slave
53                REQ    = 3'b100,      // Request bus access
54                SADDR  = 3'b101,      // Send slave device address
55                WAIT   = 3'b110,      // Wait for acknowledgement
56                SPLIT  = 3'b111;      // Wait for split slave to be ready
57
58     // State variables
59     reg [2:0] state, next_state, prev_state;
60
61     // Next state logic
62     always @(*) begin
```

```

63         case (state)
64             IDLE : next_state = (dvalid) ? REQ : IDLE;
65             REQ  : next_state = (mbgrant) ? SADDR : REQ;
66             SADDR : next_state = (counter == SLAVE_DEVICE_ADDR_WIDTH-1) ? WAIT : SADDR;
67             WAIT  : next_state = (ack) ? ADDR : ((timeout == TIMEOUT_TIME) ? IDLE : WAIT);
68             ADDR  : next_state = (counter == SLAVE_MEM_ADDR_WIDTH-1) ? ((mode) ? WDATA :
RDATA) : ADDR;
69             RDATA : next_state = (msplit) ? SPLIT : ((svalid && (counter == DATA_WIDTH-1)) ?
IDLE : RDATA);
70             WDATA : next_state = (counter == DATA_WIDTH-1) ? IDLE : WDATA;
71             SPLIT : next_state = (!msplit && mbgrant) ? RDATA : SPLIT;
72             default: next_state = IDLE;
73         endcase
74     end
75
76     // State transition logic (async reset)
77     always @(posedge clk or negedge rstn) begin
78         if (!rstn) begin
79             state <= IDLE;
80             prev_state <= IDLE;
81         end else begin
82             prev_state <= state;
83             state <= next_state;
84         end
85     end
86
87     // Combinational output assignments
88     assign dready = (state == IDLE);
89     assign drdata = rdata;
90     assign mmode  = mode;
91     assign mbreq  = (state != IDLE); // Keep bus request while master is in need of the bus
92
93     // Sequential output logic (async reset)
94     always @(posedge clk or negedge rstn) begin
95         if (!rstn) begin
96             wdata <= 'b0;
97             rdata <= 'b0;
98             addr  <= 'b0;
99             mode  <= 0;
100            counter <= 'b0;
101            mvalid <= 0;
102            mwdata <= 0;
103            timeout <= 'b0;
104        end
105        else begin
106            case (state)
107                IDLE : begin
108                    counter <= 'b0;
109                    mvalid <= 0;
110                    timeout <= 'b0;
111                    if (dvalid) begin // Have to send data
112                        wdata <= dwdata;
113                        addr  <= daddr;
114                        mode  <= dmode;
115                        // Don't clear rdata - it holds the previous read result until
overwritten
116                    end else begin
117                        wdata <= wdata;
118                        addr  <= addr;
119                        mode  <= mode;
120                    end
121                end
122
123                REQ : begin
124                    // Wait for bus grant
125                end
126
127                SADDR : begin // Send slave device address (LSB-first, bits [15:12])

```

```

128         mwdata <= addr[SLAVE_MEM_ADDR_WIDTH + counter];
129         mvalid <= 1'b1;
130         if (counter == SLAVE_DEVICE_ADDR_WIDTH-1) begin
131             counter <= 'b0;
132         end else begin
133             counter <= counter + 1;
134         end
135     end
136
137     WAIT : begin
138         mvalid <= 1'b0;
139         timeout <= timeout + 1;
140     end
141
142     ADDR : begin // Send slave mem address
143         mwdata <= addr[counter];
144         mvalid <= 1'b1;
145         if (counter == SLAVE_MEM_ADDR_WIDTH-1) begin
146             counter <= 'b0;
147         end else begin
148             counter <= counter + 1;
149         end
150     end
151
152     RDATA : begin // Receive data from slave
153         mvalid <= 1'b0;
154         if (svalid) begin
155             rdata[counter] <= mrdata;
156             if (counter == DATA_WIDTH-1) begin
157                 counter <= 'b0;
158             end else begin
159                 counter <= counter + 1;
160             end
161         end else begin
162             rdata <= rdata;
163             counter <= counter;
164         end
165     end
166
167     WDATA : begin // Send data to slave
168         // Add setup cycle delay to sync with slave's ADDR->WDATA skip cycle
169         if (prev_state == ADDR && state == WDATA) begin
170             // First cycle after ADDR->WDATA: setup time, don't transmit yet
171             mvalid <= 1'b0;
172             mwdata <= 1'b0;
173         end else begin
174             // Normal transmission
175             mwdata <= wdata[counter];
176             mvalid <= 1'b1;
177             if (counter == DATA_WIDTH-1) begin
178                 counter <= 'b0;
179             end else begin
180                 counter <= counter + 1;
181             end
182         end
183     end
184
185
186     SPLIT : begin
187         mvalid <= 1'b0;
188     end
189
190     default: begin
191         wdata <= wdata;
192         rdata <= rdata;
193         addr <= addr;
194         mode <= mode;
195         counter <= counter;

```

```

196             mvalid    <= mvalid;
197             mwdata    <= mwdata;
198             timeout   <= timeout;
199         end
200     endcase
201 end
202 end
203
204 endmodule

```

Listing 1: Master Port FSM (master_port.v)

Slave Port (slave_port.v)

```

1  `timescale 1ns / 1ps
2
3  module slave_port #(
4      parameter ADDR_WIDTH = 12,
5      parameter DATA_WIDTH = 8,
6      parameter SPLIT_EN = 0
7  )(
8      input clk,
9      input rstn,
10
11     // Signals connecting to slave memory
12     input  [DATA_WIDTH-1:0] smemrdata, // data read from the slave memory
13     input                    rvalid,   // read data is available
14     output reg               smemwen,   // write enable
15     output reg               smemren,   // read enable
16     output reg [ADDR_WIDTH-1:0] smemaddr, // slave memory address
17     output reg [DATA_WIDTH-1:0] smemwdata, // data written to the slave memory
18
19     // Signals connecting to serial bus
20     input                    swdata,    // write data and address from master
21     output reg               srdata,    // read data to the master
22     input                    smode,     // 0 - read, 1 - write
23     input                    mvalid,    // wdata valid (receiving data and address from
24     master)
25     input                    split_grant, // grant to send read data
26     output reg               svalid,    // rdata valid (sending data from slave)
27     output reg               sready,    // slave is ready for transaction
28     output reg               ssplit    // 1 - split
29 );
30
31 /* Internal signals */
32 // Registers to accept data from master and slave memory
33 reg [DATA_WIDTH-1:0] wdata; // write data from master
34 reg [ADDR_WIDTH-1:0] addr;
35 wire [DATA_WIDTH-1:0] rdata;
36 reg mode;
37
38 // Counters
39 reg [7:0] counter;
40
41 // Read data from slave memory with latency
42 localparam LATENCY = 4;
43 reg [LATENCY-1:0] rcounter;
44
45 // States
46 localparam IDLE    = 3'b000,
47             ADDR    = 3'b001, // Receive address from master
48             RDATA    = 3'b010, // Send data to master
49             WDATA    = 3'b011, // Receive data from master
50             SREADY   = 3'b101, // Slave ready
51             SPLIT    = 3'b100, // Split transaction
52             WAIT     = 3'b110, // Wait for split grant
53             RVALID   = 3'b111; // Wait for read data valid

```

```

53
54 // State variables
55 reg [2:0] state, next_state, prev_state;
56
57 // Next state logic
58 always @(*) begin
59     case (state)
60         IDLE : next_state = (mvalid) ? ADDR : IDLE;
61         ADDR : next_state = (counter == ADDR_WIDTH-1) ? ((mode) ? WDATA : SREADY) :
ADDR;
62         SREADY : next_state = (mode) ? IDLE : ((SPLIT_EN) ? SPLIT : RVALID);
63         RVALID : next_state = (rvalid) ? RDATA : RVALID;
64         SPLIT : next_state = (rcounter == LATENCY) ? WAIT : SPLIT;
65         WAIT : next_state = (split_grant) ? RDATA : WAIT;
66         RDATA : next_state = (counter == DATA_WIDTH * 2) ? IDLE : RDATA;
67         WDATA : next_state = (counter == DATA_WIDTH-1) ? SREADY : WDATA;
68         default: next_state = IDLE;
69     endcase
70 end
71
72 // State transition logic (async reset)
73 always @(posedge clk or negedge rstn) begin
74     if (!rstn) begin
75         state <= IDLE;
76         prev_state <= IDLE;
77     end else begin
78         prev_state <= state;
79         state <= next_state;
80     end
81 end
82
83 // Combinational output assignments
84 assign rdata = smemrdata;
85 assign sready = (state == IDLE);
86 assign ssplit = (state == SPLIT);
87
88 // Sequential output logic (async reset)
89 always @(posedge clk or negedge rstn) begin
90     if (!rstn) begin
91         wdata <= 'b0;
92         addr <= 'b0;
93         counter <= 'b0;
94         svalid <= 0;
95         smemren <= 0;
96         smemwen <= 0;
97         mode <= 0;
98         smemaddr <= 0;
99         smemwdata <= 0;
100        srdata <= 0;
101        rcounter <= 'b0;
102    end
103    else begin
104        case (state)
105            IDLE : begin
106                counter <= 'b0;
107                svalid <= 0;
108                smemren <= 0;
109                smemwen <= 0;
110                if (mvalid) begin
111                    mode <= smode;
112                    addr[counter] <= swdata;
113                    counter <= counter + 1;
114                end else begin
115                    addr <= addr;
116                    counter <= counter;
117                    mode <= mode;
118                end
119            end

```

```

120
121 ADDR : begin
122     svalid <= 1'b0;
123     if (mvalid) begin
124         addr[counter] <= swdata;
125         if (counter == ADDR_WIDTH-1) begin
126             counter <= 'b0;
127         end else begin
128             counter <= counter + 1;
129         end
130     end else begin
131         addr <= addr;
132         counter <= counter;
133     end
134 end
135
136 SREADY: begin
137     svalid <= 1'b0;
138     if (mode) begin
139         smemwen <= 1'b1;
140         smemwdata <= wdata;
141         smemaddr <= addr;
142     end else begin
143         smemren <= 1'b1;
144         smemaddr <= addr;
145     end
146 end
147
148 RVALID: begin
149     // Waiting for read data valid - keep smemren asserted
150     smemren <= 1'b1;
151 end
152
153 SPLIT : begin // Wait for some time - keep smemren asserted
154     rcounter <= rcounter + 1;
155     smemren <= 1'b1;
156 end
157
158 WAIT : begin // Wait until grant bus access for split transfer - keep
smemren asserted
159     rcounter <= 'b0;
160     smemren <= 1'b1;
161 end
162
163 RDATA : begin // Send data to master
164     if (counter < DATA_WIDTH * 2) begin
165         if (counter[0] == 0) begin
166             // Even counter: Load bit
167             srdata <= rdata[counter >> 1]; // counter/2 gives bit index
168             svalid <= 1'b0;
169         end else begin
170             // Odd counter: Hold bit, assert valid
171             svalid <= 1'b1;
172         end
173         smemren <= 1'b1;
174         counter <= counter + 1;
175     end else begin
176         // Transmission complete
177         svalid <= 1'b0;
178         smemren <= 1'b0;
179         counter <= 'b0;
180     end
181 end
182
183 WDATA : begin // Receive data from master
184     svalid <= 1'b0;
185     // Skip sampling on first cycle after transition from ADDR (setup time
for data)

```

```

186         if (mvalid && !(prev_state == ADDR && state == WDATA)) begin
187             wdata[counter] <= swdata;
188             if (counter == DATA_WIDTH-1) begin
189                 // DON'T set smemwen here - let SREADY do it with smemaddr at
the same time
190                 // This ensures both signals are set together via non-blocking
assignment
191                 counter <= 'b0;
192             end else begin
193                 counter <= counter + 1;
194             end
195         end else begin
196             wdata <= wdata;
197             counter <= counter;
198         end
199     end
200
201     default: begin
202         wdata <= wdata;
203         addr <= addr;
204         counter <= counter;
205         svalid <= svalid;
206         smemwen <= smemwen;
207         smemren <= smemren;
208         rcounter <= rcounter;
209     end
210 endcase
211 end
212 end
213
214 endmodule

```

Listing 2: Slave Port FSM (slave_port.v)

Arbiter (arbiter.v)

```

1  `timescale 1ns / 1ps
2
3  module arbiter (
4      input clk,
5      input rstn,
6
7      // Bus requests from 2 masters
8      input breq1,
9      input breq2,
10
11     // Slave ready signals
12     input sready1,           // slave 1 ready
13     input sready2,           // slave 2 ready
14     input sreadysp,          // split-supported slave ready
15
16     // Split signal from slave
17     input ssplit,
18
19     // Bus grant signals for 2 masters
20     output bgrant1,
21     output bgrant2,
22
23     // Master select: 0 - master 1, 1 - master 2
24     output msel,
25
26     // Split signals given to masters
27     output reg msplit1,
28     output reg msplit2,
29
30     // Grant access to continue split transaction (send back to slave)
31     output reg split_grant

```

```

32 );
33
34 // Priority based: high priority for master 1 - breq1
35 wire sready, sready_nsplrit;
36 reg [1:0] split_owner;
37
38 assign sready = sready1 & sready2 & sreadysp;
39 assign sready_nsplrit = sready1 & sready2; // non-split slaves are ready
40
41 // Split owner encoding
42 localparam NONE = 2'b00,
43             SM1 = 2'b01,
44             SM2 = 2'b10;
45
46 // States
47 localparam IDLE = 3'b000,
48             M1 = 3'b001, // M1 uses bus
49             M2 = 3'b010; // M2 uses bus
50
51 // State variables
52 reg [2:0] state, next_state;
53
54 // Next state logic
55 always @(*) begin
56     case (state)
57         IDLE : begin
58             if (!ssplit) begin // either split was released or no split was there
59                 if (split_owner == SM1)
60                     next_state = M1;
61                 else if (breq1 & sready)
62                     next_state = M1;
63                 else if (split_owner == SM2)
64                     next_state = M2;
65                 else if (breq2 & sready)
66                     next_state = M2;
67                 else
68                     next_state = IDLE;
69             end
70             else begin
71                 // One master is waiting for a split transaction, other master can
72                 continue
73                 if ((split_owner == SM1) && breq2 && sready_nsplrit)
74                     next_state = M2;
75                 else if ((split_owner == SM2) && breq1 && sready_nsplrit)
76                     next_state = M1;
77                 else
78                     next_state = IDLE;
79             end
80
81             M1 : next_state = (!breq1 | (split_owner == NONE && ssplit)) ? IDLE : M1;
82             M2 : next_state = (!breq2 | (split_owner == NONE && ssplit)) ? IDLE : M2;
83
84             default: next_state = IDLE;
85         endcase
86     end
87
88 // State transition logic (async reset)
89 always @(posedge clk or negedge rstn) begin
90     if (!rstn)
91         state <= IDLE;
92     else
93         state <= next_state;
94 end
95
96 // Combinational output assignments
97 assign bgrant1 = (state == M1);
98 assign bgrant2 = (state == M2);

```

```

99     assign msel      = (state == M2);
100
101     // Sequential output assignments for split (async reset)
102     always @(posedge clk or negedge rstn) begin
103         if (!rstn) begin
104             msplit1    <= 1'b0;
105             msplit2    <= 1'b0;
106             split_owner <= NONE;
107             split_grant <= 1'b0;
108         end
109         else begin
110             case (state)
111             M1 : begin
112                 if (split_owner == NONE && ssplit) begin
113                     msplit1    <= 1'b1;
114                     split_owner <= SM1;
115                     split_grant <= 1'b0;
116                 end else if (split_owner == SM1 && !ssplit) begin
117                     msplit1    <= 1'b0;
118                     split_owner <= NONE;
119                     split_grant <= 1'b1;
120                 end else begin
121                     msplit1    <= msplit1;
122                     split_owner <= split_owner;
123                     split_grant <= 1'b0;
124                 end
125             end
126
127             M2 : begin
128                 if (split_owner == NONE && ssplit) begin
129                     msplit2    <= 1'b1;
130                     split_owner <= SM2;
131                     split_grant <= 1'b0;
132                 end else if (split_owner == SM2 && !ssplit) begin
133                     msplit2    <= 1'b0;
134                     split_owner <= NONE;
135                     split_grant <= 1'b1;
136                 end else begin
137                     msplit2    <= msplit2;
138                     split_owner <= split_owner;
139                     split_grant <= 1'b0;
140                 end
141             end
142
143             default : begin
144                 msplit1    <= msplit1;
145                 msplit2    <= msplit2;
146                 split_owner <= split_owner;
147                 split_grant <= split_grant;
148             end
149         endcase
150     end
151 end
152
153 endmodule

```

Listing 3: Arbiter (arbiter.v)

Address Decoder (addr_decoder.v)

```

1  `timescale 1ns / 1ps
2
3  module addr_decoder #(
4      parameter ADDR_WIDTH = 16,
5      parameter DEVICE_ADDR_WIDTH = 4
6  )(
7      input clk,

```

```

8   input rstn,
9
10  // Serial data from master
11  input mwdata,           // write data bus
12  input mvalid,           // valid from master
13
14  // Split transaction signals
15  input ssplit,           // split signal from slave
16  input split_grant,      // signal from arbiter ending split
17
18  // Ready signals from slaves
19  input sready1,
20  input sready2,
21  input sready3,
22
23  // Valid signals going to slaves
24  output mvalid1,
25  output mvalid2,
26  output mvalid3,
27
28  // Slave select going to muxes
29  output reg [1:0] ssel,
30
31  // Acknowledgement going back to master
32  output ack
33 );
34
35  // Internal signals
36  reg [DEVICE_ADDR_WIDTH-1:0] slave_addr;
37  reg slave_en;           // Enable slave connection
38  wire mvalid_out;
39  wire slave_addr_valid;  // Valid slave address
40  wire [2:0] sready;
41  reg [3:0] counter;
42  reg [DEVICE_ADDR_WIDTH-1:0] split_slave_addr;
43
44  // Decoder to give the correct mvalid signals to slaves
45  dec3 mvalid_decoder (
46    .sel(ssel),
47    .en(mvalid_out),
48    .out1(mvalid1),
49    .out2(mvalid2),
50    .out3(mvalid3)
51  );
52
53  // States
54  localparam IDLE    = 2'b00,
55             ADDR     = 2'b01, // Receive address from master
56             CONNECT  = 2'b10, // Enable correct slave connection
57             WAIT     = 2'b11; // Wait for transaction complete
58
59  // State variables
60  reg [1:0] state, next_state;
61
62  // Next state logic
63  always @(*) begin
64    case (state)
65      IDLE : next_state = (mvalid) ? ADDR : ((split_grant) ? WAIT : IDLE);
66      ADDR : next_state = (counter == DEVICE_ADDR_WIDTH-1) ? CONNECT : ADDR;
67      CONNECT : next_state = (slave_addr_valid) ? ((mvalid) ? WAIT : CONNECT) : IDLE;
68      WAIT : next_state = (sready[slave_addr] | ssplit) ? IDLE : WAIT;
69      default : next_state = IDLE;
70    endcase
71  end
72
73  // State transition logic (async reset)
74  always @(posedge clk or negedge rstn) begin
75    if (!rstn)

```

```

76         state <= IDLE;
77     else
78         state <= next_state;
79 end
80
81 // Combinational assignments
82 assign mvalid_out      = mvalid & slave_en;
83 assign slave_addr_valid = (slave_addr < 3) & sready[slave_addr]; // check whether ready
84                                and valid
85 assign ack              = (state == CONNECT) & slave_addr_valid; // If address invalid
86                                , do not ack
87 assign sready           = {sready3, sready2, sready1};
88
89 // Sequential output logic (async reset)
90 always @(posedge clk or negedge rstn) begin
91     if (!rstn) begin
92         slave_addr      <= 'b0;
93         slave_en         <= 0;
94         counter          <= 'b0;
95         ssel             <= 'b0;
96         split_slave_addr <= 'b0;
97     end
98     else begin
99         case (state)
100             IDLE : begin
101                 slave_en <= 0;
102                 if (mvalid) begin // Have to send data
103                     slave_addr[0] <= mwdata;
104                     counter      <= 1;
105                 end else if (split_grant) begin
106                     slave_addr <= split_slave_addr;
107                     counter    <= 'b0;
108                 end else begin
109                     slave_addr <= slave_addr;
110                     counter    <= 'b0;
111                 end
112             end
113             //Master sends address serially - LSB First
114             ADDR : begin // Receive slave device address (LSB-first, continuing from
115 bit 0)
116                 slave_addr[counter] <= mwdata;
117                 if (counter == DEVICE_ADDR_WIDTH-1) begin
118                     counter <= 'b0;
119                 end else begin
120                     counter <= counter + 1;
121                 end
122             end
123             CONNECT : begin
124                 slave_en <= 1;
125                 ssel     <= slave_addr[1:0];
126             end
127             WAIT : begin
128                 slave_en <= 1;
129                 ssel     <= slave_addr[1:0];
130                 if (ssplit)
131                     split_slave_addr <= slave_addr;
132                 else
133                     split_slave_addr <= split_slave_addr;
134             end
135             default: begin
136                 slave_addr      <= slave_addr;
137                 slave_en         <= slave_en;
138                 counter          <= counter;
139                 ssel             <= ssel;
140                 split_slave_addr <= split_slave_addr;

```

```

141         end
142     endcase
143 end
144 end
145
146 endmodule

```

Listing 4: Address Decoder (addr_decoder.v)

Bus Bridge Master (bus_bridge_master.v)

```

1  `timescale 1ns / 1ps
2
3  module bus_bridge_master #(
4      parameter ADDR_WIDTH = 16,
5      parameter DATA_WIDTH = 8,
6      parameter SLAVE_MEM_ADDR_WIDTH = 12,
7      parameter BB_ADDR_WIDTH = 12,
8      parameter UART_CLOCKS_PER_PULSE = 5208,
9      parameter LOCAL_MEM_SIZE = 2048,      // Local memory size
10     parameter LOCAL_MEM_ADDR_WIDTH = 11    // log2(LOCAL_MEM_SIZE)
11 ) (
12     input clk, rstn,
13
14     //-----
15     // Serial Bus Interface (as a master)
16     //-----
17     input wire mrdata,      // Read data from bus
18     output wire mwdata,     // Write data and address to bus
19     output wire mmode,      // 0 - read, 1 - write
20     output wire mvalid,     // Write data valid
21     input wire svalid,      // Read data valid from slave
22
23     //-----
24     // Arbiter Interface
25     //-----
26     output wire mbreq,      // Bus request
27     input wire mbgrant,     // Bus grant
28     input wire msplit,      // Split signal
29
30     //-----
31     // Address Decoder Interface
32     //-----
33     input wire ack,         // Acknowledgement
34
35     //-----
36     // Local Memory Interface - Direct access to master's internal BRAM
37     //-----
38     input wire          lmem_wen,    // Local memory write enable
39     input wire          lmem_ren,    // Local memory read enable
40     input wire [LOCAL_MEM_ADDR_WIDTH-1:0] lmem_addr, // Local memory address
41     input wire [DATA_WIDTH-1:0] lmem_wdata, // Local memory write data
42     output wire [DATA_WIDTH-1:0] lmem_rdata, // Local memory read data
43     output wire          lmem_rvalid, // Local memory read valid
44
45     //-----
46     // Bus Bridge UART Interface (connecting to remote system)
47     //-----
48     output wire u_tx,        // UART transmit (send read data back)
49     input wire u_rx          // UART receive (receive commands)
50 );
51
52 //-----
53 // Parameters
54 //-----
55 localparam UART_RX_DATA_WIDTH = DATA_WIDTH + BB_ADDR_WIDTH + 1; // Receive: mode + data
56 + addr

```

```

56     localparam UART_TX_DATA_WIDTH = DATA_WIDTH;           // Transmit: only read
    data

57
58     //-----
59     // Signals to Master Port
60     //-----
61     reg [DATA_WIDTH-1:0]   dwdata;           // Write data to master port
62     wire [DATA_WIDTH-1:0]  drdata;           // Read data from master port
63     wire [ADDR_WIDTH-1:0]  daddr;            // Address to master port
64     reg                    dvalid;           // Transaction valid
65     wire                   dready;           // Master port ready
66     reg                    dmode;            // Transaction mode
67
68     //-----
69     // FIFO Signals
70     //-----
71     reg                    fifo_enq;
72     reg                    fifo_deq;
73     reg [UART_RX_DATA_WIDTH-1:0] fifo_din;
74     wire [UART_RX_DATA_WIDTH-1:0] fifo_dout;
75     wire                    fifo_empty;
76
77     //-----
78     // UART Signals
79     //-----
80     reg [UART_TX_DATA_WIDTH-1:0] u_din;
81     reg                    u_en;
82     wire                    u_tx_busy;
83     wire                    u_rx_ready;
84     wire [UART_RX_DATA_WIDTH-1:0] u_dout;
85
86     //-----
87     // Internal Control Signals
88     //-----
89     reg [BB_ADDR_WIDTH-1:0] bb_addr;
90     reg                    expect_rdata;
91     reg                    prev_u_ready;
92     reg                    prev_m_ready;
93
94     //-----
95     // Master Port Instantiation
96     //-----
97     master_port #(
98         .ADDR_WIDTH(ADDR_WIDTH),
99         .DATA_WIDTH(DATA_WIDTH),
100        .SLAVE_MEM_ADDR_WIDTH(SLAVE_MEM_ADDR_WIDTH)
101    ) master (
102        .clk(clk),
103        .rstn(rstn),
104        .dwdata(dwdata),
105        .drdata(drdata),
106        .daddr(daddr),
107        .dvalid(dvalid),
108        .dready(dready),
109        .dmode(dmode),
110        .mrdata(mrdata),
111        .mwdata(mwdata),
112        .mmode(mmode),
113        .mvalid(mvalid),
114        .svalid(svalid),
115        .mbreq(mbreq),
116        .mbgrant(mbgrant),
117        .msplit(msplit),
118        .ack(ack)
119    );
120
121     //-----
122     // FIFO Module Instantiation

```

```

123 //-----
124 fifo #(
125     .DATA_WIDTH(UART_RX_DATA_WIDTH),
126     .DEPTH(8)
127 ) fifo_queue (
128     .clk(clk),
129     .rstn(rstn),
130     .enq(fifo_enq),
131     .deq(fifo_deq),
132     .data_in(fifo_din),
133     .data_out(fifo_dout),
134     .empty(fifo_empty)
135 );
136
137 //-----
138 // UART Module Instantiation
139 //-----
140 uart #(
141     .CLOCKS_PER_PULSE(UART_CLOCKS_PER_PULSE),
142     .TX_DATA_WIDTH(UART_TX_DATA_WIDTH),
143     .RX_DATA_WIDTH(UART_RX_DATA_WIDTH)
144 ) uart_module (
145     .data_input(u_din),
146     .data_en(u_en),
147     .clk(clk),
148     .rstn(rstn),
149     .tx(u_tx),
150     .tx_busy(u_tx_busy),
151     .rx(u_rx),
152     .ready(u_rx_ready),
153     .data_output(u_dout)
154 );
155
156 //-----
157 // Address Converter - Maps BB address to full bus address
158 //-----
159 addr_convert #(
160     .BB_ADDR_WIDTH(BB_ADDR_WIDTH),
161     .BUS_ADDR_WIDTH(ADDR_WIDTH),
162     .BUS_MEM_ADDR_WIDTH(SLAVE_MEM_ADDR_WIDTH)
163 ) addr_convert_module (
164     .bb_addr(bb_addr),
165     .bus_addr(daddr)
166 );
167
168 //-----
169 // Local Memory BRAM Instantiation
170 //-----
171 master_memory_bram #(
172     .ADDR_WIDTH(LOCAL_MEM_ADDR_WIDTH),
173     .DATA_WIDTH(DATA_WIDTH),
174     .MEM_SIZE(LOCAL_MEM_SIZE)
175 ) local_mem (
176     .clk(clk),
177     .rstn(rstn),
178     .wen(lmem_wen),
179     .ren(lmem_ren),
180     .addr(lmem_addr),
181     .wdata(lmem_wdata),
182     .rdata(lmem_rdata),
183     .rvalid(lmem_rvalid)
184 );
185
186 //-----
187 // UART RX to FIFO Logic (async reset)
188 // Enqueue received UART data into FIFO
189 //-----
190 always @(posedge clk or negedge rstn) begin

```

```

191     if (!rstn) begin
192         fifo_din      <= {UART_RX_DATA_WIDTH{1'b0}};
193         fifo_enq      <= 1'b0;
194         prev_u_ready  <= 1'b0;
195     end
196     else begin
197         prev_u_ready <= u_rx_ready;
198
199         if (u_rx_ready && !prev_u_ready) begin
200             fifo_din <= u_dout;
201             fifo_enq <= 1'b1;
202         end
203         else begin
204             fifo_din <= fifo_din;
205             fifo_enq <= 1'b0;
206         end
207     end
208 end
209
210 //-----
211 // FIFO to Master Port Logic (async reset)
212 // Dequeue FIFO data and initiate bus transactions
213 //-----
214 always @(posedge clk or negedge rstn) begin
215     if (!rstn) begin
216         bb_addr      <= {BB_ADDR_WIDTH{1'b0}};
217         dwdata       <= {DATA_WIDTH{1'b0}};
218         dmode        <= 1'b0;
219         dvalid       <= 1'b0;
220         fifo_deq     <= 1'b0;
221         expect_rdata <= 1'b0;
222     end
223     else begin
224         if (dready && !fifo_empty && !dvalid) begin
225             bb_addr      <= fifo_dout[BB_ADDR_WIDTH-1:0];
226             dwdata       <= fifo_dout[BB_ADDR_WIDTH +: DATA_WIDTH];
227             dmode        <= fifo_dout[BB_ADDR_WIDTH + DATA_WIDTH];
228             dvalid       <= 1'b1;
229             fifo_deq     <= 1'b1;
230             expect_rdata <= ~fifo_dout[BB_ADDR_WIDTH + DATA_WIDTH]; // Expect read data
231         end
232         else begin
233             bb_addr      <= bb_addr;
234             dwdata       <= dwdata;
235             dmode        <= dmode;
236             dvalid       <= 1'b0;
237             fifo_deq     <= 1'b0;
238             expect_rdata <= expect_rdata;
239         end
240     end
241 end
242
243 //-----
244 // Master Port to UART TX Logic (async reset)
245 // Send read data back via UART after bus read completes
246 //-----
247 always @(posedge clk or negedge rstn) begin
248     if (!rstn) begin
249         u_din         <= {UART_TX_DATA_WIDTH{1'b0}};
250         u_en          <= 1'b0;
251         prev_m_ready  <= 1'b0;
252     end
253     else begin
254         prev_m_ready <= dready;
255     end
256     // Read transaction finished - send data back via UART

```

```

258         if (!prev_m_ready && dready && expect_rdata) begin
259             u_din <= drdata;
260             u_en  <= 1'b1;
261
262         end
263     else begin
264         u_din <= u_din;
265         u_en  <= 1'b0;
266     end
267 end
268 end
269
270 endmodule

```

Listing 5: Bus Bridge Master (bus_bridge_master.v)

Bus Bridge Slave (bus_bridge_slave.v)

```

1  `timescale 1ns / 1ps
2
3  module bus_bridge_slave #(
4      parameter DATA_WIDTH = 8,
5      parameter ADDR_WIDTH = 12,
6      parameter UART_CLOCKS_PER_PULSE = 5208,
7      parameter LOCAL_MEM_SIZE = 2048,      // Local memory size (half of address space)
8      parameter BRIDGE_ENABLE = 1          // Enable UART bridge functionality
9  )(
10     input clk, rstn,
11
12     //-----
13     // Serial Bus Interface (connecting to local bus)
14     //-----
15     input wire swdata,      // Write data and address from master
16     input wire smode,       // 0 - read, 1 - write, from master
17     input wire mvalid,      // wdata valid (receiving data and address from master)
18     input wire split_grant, // Grant bus access in split
19
20     output wire srdata,     // Read data to the master
21     output wire svalid,     // rdata valid (sending data from slave)
22     output wire sready,     // Slave is ready for transaction
23     output wire ssplit,     // Split signal
24
25     //-----
26     // Bus Bridge UART Interface (connecting to remote system)
27     //-----
28     output wire u_tx,       // UART transmit to remote
29     input wire u_rx         // UART receive from remote
30 );
31
32     //-----
33     // Parameters
34     //-----
35     localparam UART_TX_DATA_WIDTH = DATA_WIDTH + ADDR_WIDTH + 1; // Transmit: mode + data
36     // + addr
37     localparam UART_RX_DATA_WIDTH = DATA_WIDTH; // Receive: only read
38     data
39     localparam SPLIT_EN = 1'b1; // Enable split for UART operations
40
41     // Address bit to distinguish local vs bridge operation
42     // MSB of address: 0 = local memory, 1 = bridge to remote
43     localparam LOCAL_ADDR_MSB = ADDR_WIDTH - 1;
44
45     //-----
46     // Internal Signals - Slave Port to Controller
47     //-----
48     wire [DATA_WIDTH-1:0] sp_memrdata; // Data read from memory
49     wire sp_memwen; // Write enable from slave port

```

```

48 wire          sp_memren;          // Read enable from slave port
49 wire [ADDR_WIDTH-1:0] sp_memaddr; // Address from slave port
50 wire [DATA_WIDTH-1:0] sp_memwdata; // Write data from slave port
51 reg          sp_rvalid;          // Read valid to slave port
52 wire          sp_ready;          // Slave port ready
53
54 //-----
55 // Internal Signals - Local Memory
56 //-----
57 wire          lmem_wen;          // Local memory write enable
58 wire          lmem_ren;          // Local memory read enable
59 wire [ADDR_WIDTH-2:0] lmem_addr; // Local memory address (one bit less)
60 wire [DATA_WIDTH-1:0] lmem_rdata; // Local memory read data
61 wire          lmem_rvalid;       // Local memory read valid
62
63 //-----
64 // Internal Signals - UART
65 //-----
66 reg [UART_TX_DATA_WIDTH-1:0] u_din; // UART transmit data
67 reg          u_en;                 // UART transmit enable
68 wire          u_tx_busy;          // UART transmitter busy
69 wire          u_rx_ready;         // UART receive data ready
70 wire [UART_RX_DATA_WIDTH-1:0] u_dout; // UART receive data
71
72 //-----
73 // Internal Signals - Controller
74 //-----
75 reg [DATA_WIDTH-1:0] latched_rdata; // Latched read data from UART
76 reg [DATA_WIDTH-1:0] latched_wdata; // Latched write data from slave_port
77 reg [ADDR_WIDTH-1:0] latched_addr;  // Latched address from slave_port
78 reg          rdata_received;       // Flag: UART data received
79 reg          prev_u_rx_ready;      // Previous UART ready state
80 reg          pending_write;        // Flag: write pending for UART TX
81 reg          pending_read;         // Flag: read pending for UART TX
82 wire          is_bridge_access;    // Address indicates bridge access
83 wire          is_local_access;     // Address indicates local memory access
84
85 //-----
86 // Address Decode Logic (using latched address for FSM decisions)
87 //-----
88 wire is_bridge_access_now = sp_memaddr[LOCAL_ADDR_MSB] & BRIDGE_ENABLE;
89 wire is_local_access_now  = ~sp_memaddr[LOCAL_ADDR_MSB];
90
91 assign is_bridge_access = latched_addr[LOCAL_ADDR_MSB] & BRIDGE_ENABLE;
92 assign is_local_access  = ~latched_addr[LOCAL_ADDR_MSB];
93
94 // Local memory signals
95 assign lmem_wen  = sp_memwen & is_local_access;
96 assign lmem_ren  = sp_memren & is_local_access;
97 assign lmem_addr = sp_memaddr[ADDR_WIDTH-2:0];
98
99 //-----
100 // Slave Port Instantiation
101 // NOTE: sp_split_grant is defined later after state declaration
102 //       sp_ssplrit is internal; we extend ssplrit for bridge operations
103 //-----
104 wire sp_split_grant; // Forward declaration - assigned after state declared
105 wire sp_ssplrit;    // Internal ssplrit from slave_port
106
107 slave_port #(
108     .ADDR_WIDTH(ADDR_WIDTH),
109     .DATA_WIDTH(DATA_WIDTH),
110     .SPLIT_EN(SPLIT_EN)
111 ) slave (
112     .clk(clk),
113     .rstn(rstn),
114     .smemrdata(sp_memrdata),
115     .rvalid(sp_rvalid),

```

```

116     .smemwen(sp_memwen),
117     .smemren(sp_memren),
118     .smemaddr(sp_memaddr),
119     .smemwdata(sp_memwdata),
120     .swdata(swdata),
121     .srdata(srdata),
122     .smode(smode),
123     .mvalid(mvalid),
124     .split_grant(sp_split_grant),
125     .svalid(svalid),
126     .sready(sp_ready),
127     .ssplit(sp_ssplrit)
128 );
129
130 //-----
131 // Local Memory BRAM Instantiation
132 //-----
133 slave_memory_bram #(
134     .ADDR_WIDTH(ADDR_WIDTH-1), // One bit less for local addressing
135     .DATA_WIDTH(DATA_WIDTH),
136     .MEM_SIZE(LOCAL_MEM_SIZE)
137 ) local_mem (
138     .clk(clk),
139     .rstn(rstn),
140     .wen(lmem_wen),
141     .ren(lmem_ren),
142     .addr(lmem_addr),
143     .wdata(sp_memwdata),
144     .rdata(lmem_rdata),
145     .rvalid(lmem_rvalid)
146 );
147
148 //-----
149 // UART Module Instantiation (for bridge operations)
150 //-----
151 uart #(
152     .CLOCKS_PER_PULSE(UART_CLOCKS_PER_PULSE),
153     .TX_DATA_WIDTH(UART_TX_DATA_WIDTH),
154     .RX_DATA_WIDTH(UART_RX_DATA_WIDTH)
155 ) uart_module (
156     .data_input(u_din),
157     .data_en(u_en),
158     .clk(clk),
159     .rstn(rstn),
160     .tx(u_tx),
161     .tx_busy(u_tx_busy),
162     .rx(u_rx),
163     .ready(u_rx_ready),
164     .data_output(u_dout)
165 );
166
167 //-----
168 // Controller FSM States
169 //-----
170 localparam IDLE    = 3'b000,
171             WSEND   = 3'b001, // Send write via UART
172             RSEND   = 3'b010, // Send read request via UART
173             RDATA   = 3'b011, // Wait for UART read response
174             LOCAL   = 3'b100, // Local memory operation
175             WBUSY   = 3'b101, // Wait for UART TX busy after write
176             RBUSY   = 3'b110; // Wait for UART TX busy after read request
177
178 reg [2:0] state, next_state;
179
180 wire bridge_read_in_progress;
181 assign bridge_read_in_progress = (state == RSEND) || (state == RBUSY) || (state == RDATA);
182 );
183 assign sp_split_grant = bridge_read_in_progress ? (split_grant && rdata_received) :

```

```

split_grant;
183 assign ssplit = sp_sssplit || (bridge_read_in_progress && !rdata_received);
184 reg prev_sp_memwen, prev_sp_memren;
185
186 always @(posedge clk or negedge rstn) begin
187     if (!rstn) begin
188         latched_wdata    <= {DATA_WIDTH{1'b0}};
189         latched_addr     <= {ADDR_WIDTH{1'b0}};
190         pending_write    <= 1'b0;
191         pending_read     <= 1'b0;
192         prev_sp_memwen   <= 1'b0;
193         prev_sp_memren   <= 1'b0;
194     end else begin
195         prev_sp_memwen <= sp_memwen;
196         prev_sp_memren <= sp_memren;
197
198         // Rising edge of sp_memwen - at this clock edge, sp_memaddr should be valid
199         // because it was set by the previous clock edge's non-blocking assignment
200         if (sp_memwen && !prev_sp_memwen && state == IDLE) begin
201             latched_wdata <= sp_memwdata;
202             latched_addr  <= sp_memaddr;
203             // Check bridge access using the current address value
204             if (sp_memaddr[LOCAL_ADDR_MSB] & BRIDGE_ENABLE) begin
205                 pending_write <= 1'b1;
206
207             end else begin
208
209             end
210         end
211         // Rising edge of sp_memren
212         else if (sp_memren && !prev_sp_memren && state == IDLE) begin
213             latched_addr <= sp_memaddr;
214             if (sp_memaddr[LOCAL_ADDR_MSB] & BRIDGE_ENABLE) begin
215                 pending_read <= 1'b1;
216
217             end else begin
218
219             end
220         end
221         // Clear pending flags when entering WSEND/RSEND states
222         else if (state == WSEND || state == RSEND) begin
223             pending_write <= 1'b0;
224             pending_read  <= 1'b0;
225         end
226     end
227 end
228
229 //-----
230 // UART TX start tracking
231 //-----
232 reg uart_tx_started;
233
234 always @(posedge clk or negedge rstn) begin
235     if (!rstn)
236         uart_tx_started <= 1'b0;
237     else if (state == WSEND || state == RSEND)
238         uart_tx_started <= 1'b0; // Reset when entering busy wait states
239     else if ((state == WBUSY || state == RBUSY) && u_tx_busy)
240         uart_tx_started <= 1'b1; // Mark that TX has started
241     else if (state == IDLE)
242         uart_tx_started <= 1'b0; // Clear when back to IDLE
243 end
244
245 //-----
246 // Next State Logic (uses pending flags instead of direct sp_memwen/sp_memren)
247 // WSEND/RSEND states immediately transition to WBUSY/RBUSY to wait for UART
248 //-----
249 always @(*) begin

```

```

250     case (state)
251     IDLE: begin
252         if (pending_write)
253             next_state = WSEND;
254         else if (pending_read)
255             next_state = RSEND;
256         else if ((sp_memwen || sp_memren) && is_local_access_now)
257             next_state = LOCAL;
258         else
259             next_state = IDLE;
260     end
261     WSEND: next_state = WBUSY; // Always go to WBUSY to wait for TX to start
262     // Wait until TX has started AND completed (tx_busy goes high then low)
263     WBUSY: next_state = (uart_tx_started && !u_tx_busy) ? IDLE : WBUSY;
264     RSEND: next_state = RBUSY; // Always go to RBUSY to wait for TX to start
265     // Wait until TX has started AND completed, then wait for response
266     RBUSY: next_state = (uart_tx_started && !u_tx_busy) ? RDATA : RBUSY;
267     RDATA: next_state = (!sp_memren) ? IDLE : RDATA;
268     LOCAL: next_state = (!sp_memwen && !sp_memren) ? IDLE : LOCAL;
269     default: next_state = IDLE;
270 endcase
271 end
272
273 //-----
274 // State Transition Logic (async reset)
275 //-----
276 always @(posedge clk or negedge rstn) begin
277     if (!rstn)
278         state <= IDLE;
279     else begin
280         if (state == IDLE && (sp_memwen || sp_memren)) begin
281             end
282             if (state != next_state) begin
283                 end
284                 // Debug WBUSY state every cycle
285                 if (state == WBUSY || state == RBUSY) begin
286                     end
287                     state <= next_state;
288             end
289         end
290     end
291
292 //-----
293 // UART Receive Data Latching (async reset)
294 //-----
295 always @(posedge clk or negedge rstn) begin
296     if (!rstn) begin
297         latched_rdata <= {DATA_WIDTH{1'b0}};
298         rdata_received <= 1'b0;
299         prev_u_rx_ready <= 1'b0;
300     end
301     else begin
302         prev_u_rx_ready <= u_rx_ready;
303
304         if (state == IDLE) begin
305             rdata_received <= 1'b0;
306             latched_rdata <= {DATA_WIDTH{1'b0}};
307         end
308         else if (state == RDATA && u_rx_ready && !prev_u_rx_ready) begin
309             latched_rdata <= u_dout;
310             rdata_received <= 1'b1;
311         end
312     end
313 end
314
315 //-----
316 // UART Transmit Control (async reset) - Uses latched values
317 // Only assert u_en in WSEND/RSEND states for ONE cycle

```

```

318 //-----
319 always @(posedge clk or negedge rstn) begin
320     if (!rstn) begin
321         u_din <= {UART_TX_DATA_WIDTH{1'b0}};
322         u_en <= 1'b0;
323     end
324     else begin
325         case (state)
326             IDLE: begin
327                 u_din <= u_din;
328                 u_en <= 1'b0;
329             end
330
331             WSEND: begin
332                 // Send: {mode=1, data, addr} - full address width
333                 // Use latched values for correct data
334                 u_din <= {1'b1, latched_wdata, latched_addr};
335                 u_en <= 1'b1;
336             end
337
338             WBUSY: begin
339                 // Keep data, but deassert enable - UART should be busy now
340                 u_din <= u_din;
341                 u_en <= 1'b0;
342             end
343
344             RSEND: begin
345                 // Send: {mode=0, 0, addr} - full address width
346                 // Use latched address
347                 u_din <= {1'b0, {DATA_WIDTH{1'b0}}, latched_addr};
348                 u_en <= 1'b1;
349             end
350
351             RBUSY: begin
352                 u_din <= u_din;
353                 u_en <= 1'b0;
354             end
355
356             RDATA: begin
357                 u_din <= u_din;
358                 u_en <= 1'b0;
359             end
360
361             LOCAL: begin
362                 u_din <= u_din;
363                 u_en <= 1'b0;
364             end
365
366             default: begin
367                 u_din <= u_din;
368                 u_en <= 1'b0;
369             end
370         endcase
371     end
372 end
373
374 //-----
375 // Read Data Multiplexing and Valid Generation
376 //-----
377 // Select read data source based on access type
378 wire bridge_rvalid = (state == RDATA) && rdata_received;
379
380 // Read data mux: local memory or UART response
381 assign sp_memrdata = (state == RDATA) ? (rdata_received ? latched_rdata : u_dout) :
382     (state == LOCAL) ? lmem_rdata : {DATA_WIDTH{1'b0}};
383
384 // Read valid: either from local memory or UART response
385 always @(*) begin

```

```

386         if (state == LOCAL)
387             sp_rvalid = lmem_rvalid;
388         else if (state == RDATA)
389             sp_rvalid = rdata_received;
390         else
391             sp_rvalid = 1'b0;
392     end
393
394     //-----
395     // Ready Signal
396     //-----
397     // Slave is ready when: slave_port is ready AND not busy with UART AND state is IDLE
398     assign sready = sp_ready && !sp_memwen && !sp_memren && (state == IDLE);
399
400 endmodule

```

Listing 6: Bus Bridge Slave (bus_bridge_slave.v)

Testbench code

Top verification (master2_slave3_tb.sv)

```

1  `timescale 1ns/1ps
2
3  module master2_slave3_tb;
4
5      //-----
6      // Parameters
7      //-----
8      parameter ADDR_WIDTH = 16;
9      parameter DATA_WIDTH = 8;
10
11     // Memory Configuration:
12     // Slave 1: 2KB (2^11 = 2048 bytes), Address width = 11 bits
13     // Slave 2: 4KB (2^12 = 4096 bytes), Address width = 12 bits
14     // Slave 3: 4KB (2^12 = 4096 bytes), Address width = 12 bits, SPLIT enabled
15     parameter SLAVE1_MEM_ADDR_WIDTH = 11; // 2KB
16     parameter SLAVE2_MEM_ADDR_WIDTH = 12; // 4KB
17     parameter SLAVE3_MEM_ADDR_WIDTH = 12; // 4KB
18     parameter MAX_SLAVE_ADDR_WIDTH = 12; // Largest slave address width
19     parameter DEVICE_ADDR_WIDTH = ADDR_WIDTH - MAX_SLAVE_ADDR_WIDTH; // 16 - 12 = 4 bits
20     parameter CLK_PERIOD = 10; // 10ns = 100MHz
21
22     //-----
23     // Global Signals
24     //-----
25     reg clk;
26     reg rstn;
27
28     //-----
29     // Master 1 Device Interface
30     //-----
31     reg [DATA_WIDTH-1:0] d1_wdata;
32     wire [DATA_WIDTH-1:0] d1_rdata;
33     reg [ADDR_WIDTH-1:0] d1_addr;
34     reg d1_valid;
35     wire d1_ready;
36     reg d1_mode; // 0 - read, 1 - write
37
38     //-----
39     // Master 2 Device Interface
40     //-----
41     reg [DATA_WIDTH-1:0] d2_wdata;
42     wire [DATA_WIDTH-1:0] d2_rdata;
43     reg [ADDR_WIDTH-1:0] d2_addr;
44     reg d2_valid;
45     wire d2_ready;

```

```

46     reg                                d2_mode;                // 0 - read, 1 - write
47
48     //-----
49     // Bus Signals - Master 1
50     //-----
51     wire m1_rdata;
52     wire m1_wdata;
53     wire m1_mode;
54     wire m1_mvalid;
55     wire m1_svalid;
56     wire m1_breq;
57     wire m1_bgrant;
58     wire m1_ack;
59     wire m1_split;
60
61     //-----
62     // Bus Signals - Master 2
63     //-----
64     wire m2_rdata;
65     wire m2_wdata;
66     wire m2_mode;
67     wire m2_mvalid;
68     wire m2_svalid;
69     wire m2_breq;
70     wire m2_bgrant;
71     wire m2_ack;
72     wire m2_split;
73
74     //-----
75     // Bus Signals - Slave 1
76     //-----
77     wire s1_rdata;
78     wire s1_wdata;
79     wire s1_mode;
80     wire s1_mvalid;
81     wire s1_svalid;
82     wire s1_ready;
83
84     //-----
85     // Bus Signals - Slave 2
86     //-----
87     wire s2_rdata;
88     wire s2_wdata;
89     wire s2_mode;
90     wire s2_mvalid;
91     wire s2_svalid;
92     wire s2_ready;
93
94     //-----
95     // Bus Signals - Slave 3 (Split-capable)
96     //-----
97     wire s3_rdata;
98     wire s3_wdata;
99     wire s3_mode;
100    wire s3_mvalid;
101    wire s3_svalid;
102    wire s3_ready;
103    wire s3_split;
104
105    //-----
106    // Split Transaction Control
107    //-----
108    wire split_grant;
109
110    //-----
111    // Testbench Control Variables
112    //-----
113    integer i;

```

```

114 reg [ADDR_WIDTH-1:0] rand_addr1, rand_addr2, rand_addr3;
115 reg [DATA_WIDTH-1:0] rand_data1, rand_data2;
116 reg [DATA_WIDTH-1:0] slave_mem_data1, slave_mem_data2;
117 reg [1:0] slave_id1, slave_id2;
118 reg m1_accepted, m2_accepted; // Transaction arbitration tracking
119 reg [DATA_WIDTH-1:0] d1_rdata_before, d2_rdata_before; // For detecting transaction
    completion
120
121 // Combined slave ready signal
122 wire s_ready;
123 assign s_ready = s1_ready & s2_ready & s3_ready;
124
125 //=====
126 // DUT Instantiations
127 //=====
128
129 //-----
130 // Master Port 1
131 //-----
132 master_port #(
133     .ADDR_WIDTH(ADDR_WIDTH),
134     .DATA_WIDTH(DATA_WIDTH),
135     .SLAVE_MEM_ADDR_WIDTH(MAX_SLAVE_ADDR_WIDTH)
136 ) master1 (
137     .clk(clk),
138     .rstn(rstn),
139     .dwdata(d1_wdata),
140     .drdata(d1_rdata),
141     .daddr(d1_addr),
142     .dvalid(d1_valid),
143     .dready(d1_ready),
144     .dmode(d1_mode),
145     .mrdata(m1_rdata),
146     .mwdata(m1_wdata),
147     .mmode(m1_mode),
148     .mvalid(m1_mvalid),
149     .svalid(m1_svalid),
150     .mbreq(m1_breq),
151     .mbgrant(m1_bgrant),
152     .ack(m1_ack),
153     .msplit(m1_split)
154 );
155
156 //-----
157 // Master Port 2
158 //-----
159 master_port #(
160     .ADDR_WIDTH(ADDR_WIDTH),
161     .DATA_WIDTH(DATA_WIDTH),
162     .SLAVE_MEM_ADDR_WIDTH(MAX_SLAVE_ADDR_WIDTH)
163 ) master2 (
164     .clk(clk),
165     .rstn(rstn),
166     .dwdata(d2_wdata),
167     .drdata(d2_rdata),
168     .daddr(d2_addr),
169     .dvalid(d2_valid),
170     .dready(d2_ready),
171     .dmode(d2_mode),
172     .mrdata(m2_rdata),
173     .mwdata(m2_wdata),
174     .mmode(m2_mode),
175     .mvalid(m2_mvalid),
176     .svalid(m2_svalid),
177     .mbreq(m2_breq),
178     .mbgrant(m2_bgrant),
179     .ack(m2_ack),
180     .msplit(m2_split)

```

```

181 );
182
183 //-----
184 // Slave 1 (2KB, No Split)
185 //-----
186 slave #(
187     .ADDR_WIDTH(SLAVE1_MEM_ADDR_WIDTH),
188     .DATA_WIDTH(DATA_WIDTH),
189     .SPLIT_EN(0),
190     .MEM_SIZE(2048)
191 ) slave1 (
192     .clk(clk),
193     .rstn(rstn),
194     .srdata(s1_rdata),
195     .swdata(s1_wdata),
196     .smode(s1_mode),
197     .svalid(s1_svalid),
198     .mvalid(s1_mvalid),
199     .sready(s1_ready),
200     .ssplit(), // Not used
201     .split_grant(1'b0)
202 );
203
204 //-----
205 // Slave 2 (4KB, No Split)
206 //-----
207 slave #(
208     .ADDR_WIDTH(SLAVE2_MEM_ADDR_WIDTH),
209     .DATA_WIDTH(DATA_WIDTH),
210     .SPLIT_EN(0),
211     .MEM_SIZE(4096)
212 ) slave2 (
213     .clk(clk),
214     .rstn(rstn),
215     .srdata(s2_rdata),
216     .swdata(s2_wdata),
217     .smode(s2_mode),
218     .svalid(s2_svalid),
219     .mvalid(s2_mvalid),
220     .sready(s2_ready),
221     .ssplit(), // Not used
222     .split_grant(1'b0)
223 );
224
225 //-----
226 // Slave 3 (4KB, Split Enabled)
227 //-----
228 slave #(
229     .ADDR_WIDTH(SLAVE3_MEM_ADDR_WIDTH),
230     .DATA_WIDTH(DATA_WIDTH),
231     .SPLIT_EN(1),
232     .MEM_SIZE(4096)
233 ) slave3 (
234     .clk(clk),
235     .rstn(rstn),
236     .srdata(s3_rdata),
237     .swdata(s3_wdata),
238     .smode(s3_mode),
239     .svalid(s3_svalid),
240     .mvalid(s3_mvalid),
241     .sready(s3_ready),
242     .ssplit(s3_split),
243     .split_grant(split_grant)
244 );
245
246 //-----
247 // Bus Interconnect
248 //-----

```

```

249 bus_m2_s3 #(
250     .ADDR_WIDTH(ADDR_WIDTH),
251     .DATA_WIDTH(DATA_WIDTH),
252     .SLAVE1_MEM_ADDR_WIDTH(SLAVE1_MEM_ADDR_WIDTH),
253     .SLAVE2_MEM_ADDR_WIDTH(SLAVE2_MEM_ADDR_WIDTH),
254     .SLAVE3_MEM_ADDR_WIDTH(SLAVE3_MEM_ADDR_WIDTH)
255 ) bus (
256     .clk(clk),
257     .rstn(rstn),
258
259     // Master 1 connections
260     .m1_rdata(m1_rdata),
261     .m1_wdata(m1_wdata),
262     .m1_mode(m1_mode),
263     .m1_mvalid(m1_mvalid),
264     .m1_svalid(m1_svalid),
265     .m1_breq(m1_breq),
266     .m1_bgrant(m1_bgrant),
267     .m1_ack(m1_ack),
268     .m1_split(m1_split),
269
270     // Master 2 connections
271     .m2_rdata(m2_rdata),
272     .m2_wdata(m2_wdata),
273     .m2_mode(m2_mode),
274     .m2_mvalid(m2_mvalid),
275     .m2_svalid(m2_svalid),
276     .m2_breq(m2_breq),
277     .m2_bgrant(m2_bgrant),
278     .m2_ack(m2_ack),
279     .m2_split(m2_split),
280
281     // Slave 1 connections
282     .s1_rdata(s1_rdata),
283     .s1_wdata(s1_wdata),
284     .s1_mode(s1_mode),
285     .s1_mvalid(s1_mvalid),
286     .s1_svalid(s1_svalid),
287     .s1_ready(s1_ready),
288
289     // Slave 2 connections
290     .s2_rdata(s2_rdata),
291     .s2_wdata(s2_wdata),
292     .s2_mode(s2_mode),
293     .s2_mvalid(s2_mvalid),
294     .s2_svalid(s2_svalid),
295     .s2_ready(s2_ready),
296
297     // Slave 3 connections
298     .s3_rdata(s3_rdata),
299     .s3_wdata(s3_wdata),
300     .s3_mode(s3_mode),
301     .s3_mvalid(s3_mvalid),
302     .s3_svalid(s3_svalid),
303     .s3_ready(s3_ready),
304     .s3_split(s3_split),
305
306     .split_grant(split_grant)
307 );
308
309 //=====
310 // Clock Generation
311 //=====
312 initial begin
313     clk = 0;
314     forever #(CLK_PERIOD/2) clk = ~clk;
315 end
316

```

```

317 //=====
318 // Task: Random Delay
319 //=====
320 task random_delay;
321     integer delay;
322     begin
323         delay = $urandom % 10; // Random delay 0-9 clock cycles
324         $display("Random delay: %0d cycles", delay);
325         repeat(delay) @(posedge clk);
326     end
327 endtask
328
329 //=====
330 // Main Test Stimulus
331 //=====
332 initial begin
333     // Waveform dump for debugging
334     $dumpfile("master2_slave3_tb.vcd");
335     $dumpvars(0, master2_slave3_tb);
336
337     // Initialize signals
338     rstn = 0;
339     d1_valid = 0;
340     d1_wdata = 8'b0;
341     d1_addr = 16'b0;
342     d1_mode = 0;
343     d2_valid = 0;
344     d2_wdata = 8'b0;
345     d2_addr = 16'b0;
346     d2_mode = 0;
347
348     // Reset sequence
349     repeat(3) @(posedge clk);
350     rstn = 1;
351     $display("\n== ADS Bus System Test Started ==\n");
352     repeat(2) @(posedge clk);
353
354     //=====
355     // Test Loop: 20 iterations of random transactions
356     //=====
357     for (i = 0; i < 20; i = i + 1) begin
358         $display("\n--- Iteration %0d ---", i);
359
360         // Generate random addresses and data according to memory map
361         // Memory Map (16-bit addresses):
362         //   Slave 1 (2KB): 0x0000-0x07FF (device ID 0x0, 11-bit addr)
363         //   Slave 2 (4KB): 0x1000-0x1FFF (device ID 0x1, 12-bit addr)
364         //   Slave 3 (4KB): 0x2000-0x2FFF (device ID 0x2, 12-bit addr)
365
366         // Generate address 1: randomly select slave and address within range
367         slave_id1 = $urandom % 3; // 0, 1, or 2
368         case (slave_id1)
369             2'b00: rand_addr1 = 16'h0000 + ($urandom % 16'h0800); // Slave 1: 0x0000-0
x07FF (2KB)
370             2'b01: rand_addr1 = 16'h1000 + ($urandom % 16'h1000); // Slave 2: 0x1000-0
x1FFF (4KB)
371             2'b10: rand_addr1 = 16'h2000 + ($urandom % 16'h1000); // Slave 3: 0x2000-0
x2FFF (4KB)
372             default: rand_addr1 = 16'h0000;
373         endcase
374         rand_data1 = $urandom;
375
376         // Generate address 2: randomly select slave and address within range
377         slave_id2 = $urandom % 3; // 0, 1, or 2
378         case (slave_id2)
379             2'b00: rand_addr2 = 16'h0000 + ($urandom % 16'h0800); // Slave 1: 0x0000-0
x07FF (2KB)
380             2'b01: rand_addr2 = 16'h1000 + ($urandom % 16'h1000); // Slave 2: 0x1000-0

```

```

x1FFF (4KB)
381      2'b10: rand_addr2 = 16'h2000 + ($urandom % 16'h1000); // Slave 3: 0x2000-0
x2FFF (4KB)
382      default: rand_addr2 = 16'h0000;
383      endcase
384      rand_data2 = $urandom;
385
386      //=====
387      // Test 1: Sequential Write Transactions (M1 then M2)
388      //=====
389      $display("Generated: M1 addr=0x%0h data=0x%0h, M2 addr=0x%0h data=0x%0h",
390      rand_addr1, rand_data1[DATA_WIDTH-1:0], rand_addr2, rand_data2[
DATA_WIDTH-1:0]);
391      wait (d1_ready == 1 && d2_ready == 1 && s_ready == 1);
392      @(posedge clk);
393
394      // Master 1 write request
395      d1_addr = rand_addr1[ADDR_WIDTH-1:0];
396      d1_wdata = rand_data1[DATA_WIDTH-1:0];
397      d1_mode = 1; // Write
398      d1_valid = 1;
399
400      random_delay();
401
402      // Master 2 write request (tests arbitration)
403      @(posedge clk);
404      d2_addr = rand_addr2[ADDR_WIDTH-1:0];
405      d2_wdata = rand_data2[DATA_WIDTH-1:0];
406      d2_mode = 1; // Write
407      d2_valid = 1;
408
409      @(posedge clk);
410      d1_valid = 0;
411      d2_valid = 0;
412
413      // Wait for transactions to complete (fixed: wait for LOW then HIGH)
414      wait (d1_ready == 0 || d2_ready == 0); // Wait for transaction to start
415      wait (d1_ready == 1 && d2_ready == 1 && s_ready == 1); // Wait for all to
complete
416      repeat(2) @(posedge clk);
417
418      // Verify Master 1 write
419      if (slave_id1 == 2'b00)
420          slave_mem_data1 = slave1.sm.memory[d1_addr[SLAVE1_MEM_ADDR_WIDTH-1:0]];
421      else if (slave_id1 == 2'b01)
422          slave_mem_data1 = slave2.sm.memory[d1_addr[SLAVE2_MEM_ADDR_WIDTH-1:0]];
423      else if (slave_id1 == 2'b10)
424          slave_mem_data1 = slave3.sm.memory[d1_addr[SLAVE3_MEM_ADDR_WIDTH-1:0]];
425
426      if (slave_id1 != 2'b11 && slave_mem_data1 != d1_wdata) begin
427          $display("ERROR: Master 1 write failed - Addr: 0x%0h, Expected: 0x%0h, Got:
0x%0h",
428                  d1_addr, d1_wdata, slave_mem_data1);
429      end else if (slave_id1 != 2'b11) begin
430          $display("PASS: Master 1 write to 0x%0h successful", d1_addr);
431      end
432
433      // Verify Master 2 write
434      if (slave_id2 == 2'b00)
435          slave_mem_data2 = slave1.sm.memory[d2_addr[SLAVE1_MEM_ADDR_WIDTH-1:0]];
436      else if (slave_id2 == 2'b01)
437          slave_mem_data2 = slave2.sm.memory[d2_addr[SLAVE2_MEM_ADDR_WIDTH-1:0]];
438      else if (slave_id2 == 2'b10)
439          slave_mem_data2 = slave3.sm.memory[d2_addr[SLAVE3_MEM_ADDR_WIDTH-1:0]];
440
441      if (slave_id2 != 2'b11 && slave_mem_data2 != d2_wdata) begin
442          $display("ERROR: Master 2 write failed - Addr: 0x%0h, Expected: 0x%0h, Got:
0x%0h",

```

```

443         d2_addr, d2_wdata, slave_mem_data2);
444     end else if (slave_id2 != 2'b11) begin
445         $display("PASS: Master 2 write to 0x%0h successful", d2_addr);
446     end
447
448     //=====
449     // Test 2: Simultaneous Read Transactions (Priority Test)
450     //=====
451     // Record initial rdata values to detect if transaction completed
452     @(posedge clk);
453     d1_rdata_before = d1_rdata;
454     d2_rdata_before = d2_rdata;
455     d1_mode = 0; // Read
456     d1_valid = 1;
457     d2_mode = 0; // Read
458     d2_valid = 1;
459
460     @(posedge clk);
461     d1_valid = 0;
462     d2_valid = 0;
463
464     // Wait for at least one transaction to start
465     $display("Test 2: Waiting for transaction to start...");
466     wait (d1_ready == 0 || d2_ready == 0);
467     $display("Test 2: Transaction started, waiting for completion...");
468
469     // Wait for all transactions to complete or be denied
470     wait (d1_ready == 1 && d2_ready == 1 && s_ready == 1);
471     $display("Test 2: Transactions complete");
472     repeat(2) @(posedge clk);
473
474     // Transaction was accepted if rdata changed from initial value
475     // (This works because read transactions update rdata)
476     m1_accepted = (d1_rdata != d1_rdata_before);
477     m2_accepted = (d2_rdata != d2_rdata_before);
478
479     // Verify Master 1 read (only if transaction was accepted)
480     if (m1_accepted) begin
481         if (slave_id1 != 2'b11 && d1_wdata != d1_rdata) begin
482             $display("ERROR: Master 1 read failed - Addr: 0x%0h, Expected: 0x%0h,
Got: 0x%0h",
483                 d1_addr, d1_wdata, d1_rdata);
484         end else if (slave_id1 != 2'b11) begin
485             $display("PASS: Master 1 read from 0x%0h successful", d1_addr);
486         end
487     end else if (slave_id1 != 2'b11) begin
488         $display("INFO: Master 1 read denied by arbiter (addr: 0x%0h)", d1_addr);
489     end
490
491     // Verify Master 2 read (only if transaction was accepted)
492     if (m2_accepted) begin
493         if (slave_id2 != 2'b11 && d2_wdata != d2_rdata) begin
494             $display("ERROR: Master 2 read failed - Addr: 0x%0h, Expected: 0x%0h,
Got: 0x%0h",
495                 d2_addr, d2_wdata, d2_rdata);
496         end else if (slave_id2 != 2'b11) begin
497             $display("PASS: Master 2 read from 0x%0h successful", d2_addr);
498         end
499     end else if (slave_id2 != 2'b11) begin
500         $display("INFO: Master 2 read denied by arbiter (addr: 0x%0h)", d2_addr);
501     end
502
503     //=====
504     // Test 3: Write-then-Read Conflict (M2 write, M1 read)
505     //=====
506     $display("Starting Test 3: Write-Read Conflict");
507     // Generate address 3: randomly select slave and address within range
508     slave_id1 = $urandom % 3; // 0, 1, or 2

```

```

509         case (slave_id1)
510             2'b00: rand_addr3 = 16'h0000 + ($urandom % 16'h0800); // Slave 1: 0x0000-0
x07FF (2KB)
511             2'b01: rand_addr3 = 16'h1000 + ($urandom % 16'h1000); // Slave 2: 0x1000-0
x1FFF (4KB)
512             2'b10: rand_addr3 = 16'h2000 + ($urandom % 16'h1000); // Slave 3: 0x2000-0
x2FFF (4KB)
513             default: rand_addr3 = 16'h0000;
514         endcase
515
516         @(posedge clk);
517         d2_addr = rand_addr3;
518         d2_wdata = rand_data1 + rand_data2;
519         d2_mode = 1; // Write
520         d2_valid = 1;
521         $display("TEST3: M2 WRITE addr=0x%0h, wdata=0x%0h", d2_addr, d2_wdata);
522
523         random_delay();
524
525         @(posedge clk);
526         d1_addr = d2_addr;
527         d1_mode = 0; // Read
528         d1_valid = 1;
529         $display("TEST3: M1 READ addr=0x%0h (should match M2 write addr)", d1_addr);
530
531         @(posedge clk);
532         d1_valid = 0;
533         d2_valid = 0;
534         $display("TEST3: Transactions issued, waiting for completion...");
535         $display("TEST3: Initial state - d1_ready=%b, d2_ready=%b, s_ready=%b", d1_ready
, d2_ready, s_ready);
536
537         // Wait for transactions to complete (fixed: wait for LOW then HIGH)
538         $display("TEST3: Waiting for transaction to start (ready to go low)...");
539         wait (d1_ready == 0 || d2_ready == 0); // Wait for transaction to start
540         $display("TEST3: Transaction started, waiting for all to complete...");
541         wait (d1_ready == 1 && d2_ready == 1 && s_ready == 1); // Wait for all to
complete
542         $display("TEST3: All transactions complete");
543         repeat(2) @(posedge clk);
544
545         // Verify write
546         if (slave_id1 == 2'b00)
547             slave_mem_data1 = slave1.sm.memory[d2_addr[SLAVE1_MEM_ADDR_WIDTH-1:0]];
548         else if (slave_id1 == 2'b01)
549             slave_mem_data1 = slave2.sm.memory[d2_addr[SLAVE2_MEM_ADDR_WIDTH-1:0]];
550         else if (slave_id1 == 2'b10)
551             slave_mem_data1 = slave3.sm.memory[d2_addr[SLAVE3_MEM_ADDR_WIDTH-1:0]];
552
553         $display("TEST3 VERIFY: d2_addr=0x%0h, d2_wdata=0x%0h, slave_mem_data=0x%0h,
d1_rdata=0x%0h",
554             d2_addr, d2_wdata, slave_mem_data1, d1_rdata);
555
556         if (slave_id1 != 2'b11 && slave_mem_data1 != d2_wdata) begin
557             $display("ERROR: Write-Read conflict test failed at write phase - Memory has
0x%0h, expected 0x%0h", slave_mem_data1, d2_wdata);
558         end else if (slave_id1 != 2'b11) begin
559             $display("PASS: Write-Read conflict test successful");
560         end
561     end
562
563     //=====
564     // Test Completion
565     //=====
566     repeat(10) @(posedge clk);
567     $display("\n=== All Tests Completed ===\n");
568     $finish;
569 end

```

```

570
571 //=====
572 // Timeout Watchdog (prevents infinite simulation)
573 //=====
574 initial begin
575     #1000000; // 1ms timeout
576     $display("\nERROR: Simulation timeout!");
577     $finish;
578 end
579
580 endmodule

```

Listing 7: Top verification testbench (master2_slave3_tb.sv)

Additional Resource

Github Link : <https://github.com/PrabathBK/Serial-System-Bus>